

INDIVIDUAL ASSIGNMENT

TECHNOLOGY PARK MALAYSIA

CT073-3-2-CSLLT

COMPUTER SYSTEM LOW LEVEL TECHNIQUES

APU2F2602CS

HAND OUT DATE: 1 September 2026

WEIGHTAGE: 60%

INSTRUCTIONS TO CANDIDATES:

1. Students are advised to underpin their answers with the use of references (cited using the American Psychological Association (APA) Referencing).
2. Late submissions will be awarded zero (0) unless Extenuating Circumstances (EC) are upheld.
3. Cases of plagiarism will be penalized.
4. Submit the assignment to the APU Learning Management System.
5. You must obtain 50% overall to pass this module.

Student Details

No.	Name	TP Number
1	Muhammad Hassan	TP088111

Item	Details
Intake Code	APU2F2602CS
Module	CT073-3-2-CSLLT Computer System Low Level Techniques
Project Title	SmartSignal: Real-Time Traffic Light Countdown Simulator
Assembler / Platform	NASM, x86 (32-bit) Linux

Table of Contents

1.0 Introduction 5
1.1 Introduction to Assembly Language 5
1.2 The x86 Architecture, NASM and Linux System Calls 5
1.3 Project Overview: SmartSignal 6
1.4 Assumptions 7
2.0 Research and Analysis 8
2.1 Why Assembly Language Is Still Relevant 8
2.2 Areas Where Assembly Language Is Still in Use 8
2.3 Low-Level Languages in Cyber Security, Digital Forensics and Malware Analysis 9
2.4 Practical Reverse Engineering Demonstration 11
2.5 Critical Analysis 14
3.0 System Design 16
3.1 System Functionalities 16
3.2 Signal Phase Model 16
3.3 Main Program Flowchart 17
3.4 Live Simulation Flowchart 19
3.5 Pedestrian and Emergency Event Handling 21
3.6 Configuration and Input Validation Flowchart 22
3.7 Data Design 23
4.0 System Implementation 25
4.1 Program Structure and Modular Design 25
4.2 Arithmetic Operations (ADD and SUB) 25
4.3 Loops 26
4.4 Conditional and Unconditional Jumps 28
4.5 Real-Time Countdown Projection 30
4.6 Non-Blocking Keyboard Input 31
4.7 Input Validation and Robustness 32
4.8 Testing 33
5.0 User Manual 35
5.1 System Requirements 35
5.2 Installation 35
5.3 Building and Running 35

5.4 Using the Main Menu 36
5.5 Running a Live Simulation 36
5.6 Night Mode 36
5.7 Selecting a Traffic Profile 37
5.8 Configuring Signal Timings 37
5.9 Viewing the Signal Timing Plan 37
5.10 Viewing Session Statistics 37
5.11 Exiting the Program 37
5.12 Troubleshooting 37
6.0 System Screenshots 39
7.0 Conclusion 49
8.0 References 50
Appendix A: AI Usage Declaration 52

1.0 Introduction

1.1 Introduction to Assembly Language

Assembly language is classified as a low-level language since its syntax is based on the basic commands understood by the computer processor for its functioning. It is in contrast to binary commands like B8 04 00 00 00 since the programmer uses mnemonics such as mov eax, 4. An assembler is used for converting them into machine code (Irvine, 2019). There is a one-to-one correspondence between machine language and assembly language for the ease of programmers to work directly with data, registers, memory addresses and interrupt sources which are hidden in high-level languages.

The symbolic notion of machine instructions was introduced when computers started storing programs in memory. Kathleen Booth created a “contracted language” while working on programming the ARC2 and SEC machines which later evolved into what we now call assembly language (Proven, 2022). Since then, the idea of assembly market across different processors has expanded starting with 8-bit microcontroller in home appliances and to 64-bit cores in data-centres. Even modern compilers for C, C++ and Rust programming languages translate the source program into assembly code before making an executable file that can be viewed with programs like Compiler Explorer (Godbolt,n.d.).

Assembly language is specific to the architecture. A program that is created for Intel x86 will not run on an ARM processor as is. Assembly language is very verbose. Thus, the task that requires one line in Python to execute, such as printing a certain number, involves forming a loop to create digits from the numeric value to be displayed and making a system call to display the digits. At the same time, assembly language gives very accurate control of the precise timings, memory, and execution process which is why it is still relevant in embedded systems, operating system kernels, performance-intensive libraries, and security research (Duntemann, 2009).

1.2 The x86 Architecture, NASM and Linux System Calls

SmartSignal was developed for IA-32 (Intel x86, 32-bit) using Netwide Assembler (NASM). The IA-32 architecture has 8 general-purpose registers of 32-bit, a flag register that stores the

results of the arithmetic and logical operations, and instruction pointer that stores the address of the next instruction (Intel Corporation, n.d.). Table 1 shows the register usage in the project.

Table 1: Registers used in SmartSignal

Register	Typical role in SmartSignal
EAX	System call number, return values and arithmetic results such as countdown sums
EBX	First system call argument (file descriptor); phase index while looping over tables
ECX	Loop counter for the LOOP instruction; second system call argument (buffer address)
EDX	Third system call argument (length); remainder after DIV
ESI	Source pointer to strings and to entries of the settings and profile tables
EDI	Destination pointer (countdown bar buffer) and pointer to an approach's colour table
EBP	Holds a saved value such as the colour code of a countdown bar or a computed red time
ESP	Stack pointer used by CALL/RET, PUSH/POP and PUSHAD/POPAD
EFLAGS	Zero, carry and sign flags tested by conditional jumps such as JE, JB, JC and JG

In creating programs in NASM, Intel syntax is employed, so that the operand which serves as the destination comes first; for instance, when performing add eax, ebx, it means adding EBX to EAX, putting the result into EAX. There are sections in a NASM program, namely .data for initialised variables or strings, .bss for reserved but not initialised blocks, .text for the actual code or instructions (The NASM Development Team, n.d.). The program SmartSignal operates on Linux and makes calls to the operating system using system calls. Specifically, being in the context of 32-bit Linux, one performs a system call by placing a call number in EAX and providing other parameters in EBX, ECX and EDX registers before executing a software interrupt called int 0x80 (Kerrisk, 2010). Table 2 contains all system calls made by the program.

Table 2: Linux system calls used by SmartSignal

No.	Name	Purpose in SmartSignal
1	exit	Terminates the program
3	read	Reads menu input and single key presses
4	write	Prints text and ANSI colour and cursor-control sequences
48	signal	Installs a Ctrl+C handler that restores the terminal before exiting
54	ioctl	Reads and changes terminal settings (non-canonical, no-echo mode)
162	nanosleep	Pauses for 100 ms (20 ms in fast mode) between keyboard checks
168	poll	Checks whether a key is waiting without blocking the countdown

1.3 Project Overview: SmartSignal

N Navigation applications are now showing drivers the number of seconds left until traffic lights go from red to green. For example, AMap in China shows live countdowns before the traffic lights go green using connected traffic infrastructure data (Renesis, 2026), while MapmyIndia's Mappls app has added a similar countdown at the road lights in Bengaluru in 2025 (DH Web Desk, 2025). SmartSignal is the simulator of the controller logic behind such a display. It demonstrates the operation of the four-way junction with North-South and East-West approaches and shows the current colour of the lights per approach, time left until colour change and the countdown bar that shrinks all the time.

The goals of the system are:

- To simulate six phase traffic light operation (green and amber colours and all red clearance) along with per second countdown.
- To count down until each phase comes (including pedestrian and emergency vehicle waiting time).
- To allow the operator to issue pedestrian traffic request (key P) and emergency vehicle preemption (key E).
- To provide the possibility to enter signal settings with valid input and provide with ready templates of traffic profiles.

To calculate and display the signal timing plan and session statistics using arithmetic instructions.

To demonstrate the required constructs, namely ADD and SUB arithmetic, loops, and conditional and unconditional jumps, in clean and modular NASM code.

1.4 Assumptions

1. At the intersection, North-South and East-West approaches conflict with each other.
2. The simulated second consists of ten delays of 100ms each. The drift due to the screen redrawing process can be neglected in modeling.
3. A pedestrian request at the green phase will shorten the green time to 5 seconds so that pedestrians do not wait all through the green.

4. Pedestrians cross during the all-red phase when all the movements are stopped.
5. The emergency vehicle override makes the current green stop but keeps green amber for safety and then puts all movements red for an additional 8 seconds. Emergency vehicles have priority over pedestrian requests.
6. Night mode simulates low traffic periods when all approaches show flashing amber.
7. All timing intervals are integer values that fall within realistic ranges (for example, 5-60 seconds for green and 2-6 seconds for amber) that are consistent with the change and clearance intervals as described in the Signal Timing Manual (National Academies of Sciences, Engineering, and Medicine, 2015).
8. The program runs in a Linux terminal that supports ANSI escape sequences, such as the default Ubuntu Terminal.

2.0 Research and Analysis

2.1 Why Assembly Language Is Still Relevant

High-level programming languages along with optimizing compilers can produce efficient code for almost all software applications, which is why there are only a few complete software programs developed in assembly language today. However, this programming languages remains significant for three reasons. For instance, there are operations that are impossible to convey using portable high-level programming code, which includes switching the processor between privilege levels, interrupt processing, stack preparation before any C code execution, and using special instructions such as SIMD vector operations (Intel Corporation, n.d.). Additionally, in cases where every clock cycle or every byte of data is critical, skilled programmers can write assembly code that is more efficient than what the compiler can produce. Lastly, and perhaps the most important point, every compiled program becomes machine code at the end. A person who needs to understand the software functioning without the source code (for example, a malware researcher or a forensic expert) must understand assembly language coding (Eilam, 2005).

2.2 Areas Where Assembly Language Is Still in Use

2.2.1 Embedded and Real-Time Systems

High-level programming languages along with optimizing compilers can produce efficient code for almost all software applications, which is why there are only a few complete software programs developed in assembly language today. However, this programming languages remains significant for three reasons. For instance, there are operations that are impossible to convey using portable high-level programming code, which includes switching the processor between privilege levels, interrupt processing, stack preparation before any C code execution, and using special instructions such as SIMD vector operations (Intel Corporation, n.d.).

Additionally, in cases where every clock cycle or every byte of data is critical, skilled programmers can write assembly code that is more efficient than what the compiler can produce. Lastly, and perhaps the most important point, every compiled program becomes machine code at the end. A person who needs to understand the software functioning without the source code (for example, a malware researcher or a forensic expert) must understand assembly language coding (Eilam, 2005).

2.2.2 Operating Systems, Bootloaders and Device Drivers

Kernels contain assembly, which means the junction point between the software and hardware. The Linux kernel consists of assembly code for x86 entering code that executes the system calls and interrupts, and exceptions by saving and restoring the processor registers at different privilege levels (The Linux Kernel Organization, n.d.). Bootloaders also start their execution in assembly. The lack of stack or runtime environment at the power-on stage requires using assembly language. The switches of contexts among processes, the enabling/disabling of interrupts, or the configuration of memory hardware demands privileged instructions accessed through assembly code. SmartSignal continues this path from the user end: each time the program executes int 0x80, it makes its way to the kernel entry point.

2.2.3 High-Performance Multimedia Processing

Both audio and video codecs perform the same process on millions of samples or pixels, and hence they use SIMD (single instruction, multiple data) instructions which allow many values to be processed with one action. The FFmpeg project, which accounts for a huge amount of video processing, still has manually written assembly instructions for the most complicated operations. In 2024, the developers reported that the new manually written AVX-512 code works 94 times faster than the simple C alternative, though the gain for a full encoding task was not that significant (Shilov, 2024). The developers have even published their own assembly lessons for training beginners (FFmpeg, n.d.), which indicates that the knowledge of this topic is still useful.

2.2.4 Compilers, Debugging and Performance Tuning

Even developers that never write assembly manually still have to read it. Compiler developers check the generated assembly to see if the optimizations were done successfully, and they can use Compiler Explorer to see what the line of C, C++, or Rust code translates into (Godbolt, n.d.). In case of the program crashes and does not provide any error, the debugging process shows the instruction that caused the error in assembly along with register values. Understanding the language is therefore essential for diagnosing low-level bugs and performance problems such as unexpected branches or poor memory access patterns.

2.3 Low-Level Languages in Cyber Security, Digital Forensics and Malware Analysis

Assembly language is utilized by cyber security and digital forensics experts less as a programming language and more as a reading acumen. This is due to the fact that malware is seldom delivered together with its source code, and the cyber attackers intentionally eliminate symbol information, encrypt and obfuscate their code. The only valid indication of malware functionality is its code, thus analysts have to decodify it in order to gain the necessary insight into malware functioning (Sikorski & Honig, 2012).

2.3.1 Malware Analysis

There are two types of malware investigation: static and dynamic analysis. The first type involves analysis of a file without execution, determining its format and architecture type, computing hashes, and retrieving readable strings and external functions. Advanced static analysis presumes the use of disassembler to load programs in order to analyze control flow and discover operations, which allow persistence or data transferring (Sikorski & Honig, 2012). Dynamic analysis consists of running the program in a specialized environment and following its operations with debugger. In all cases the analysts must be able to recognize assembly syntax. For instance, if the buffer bytes have been XORed with a certain key, it can be interpreted as a simple operation of decrypting the string.

2.3.2 Reverse Engineering Tools

There are tools that can translate machine code into human-readable assembly. The command-line programs such as objdump, readelf and strings are available in every Linux operating system and were used for the practical demonstration discussed in Section 2.4. Interactive disassemblers make progress by creating control flow graphs and translating assembly into C-like pseudocode. An important tool developed in this field is Ghidra, the reverse engineering tool created by the US National Security Agency and released free of charge during the RSA Conference in 2019 (National Security Agency, 2019). In less than four years it achieved over one million downloads and gained popularity both in universities and large technology and security corporations (National Security Agency, 2023). However, the presence of decompilers does not eliminate the need for comparing the output to the real assembly output since the decompiler results may not be accurate due to possible obfuscation of the code or if the code is created manually (Eilam, 2005).

2.3.3 Case Studies

Stuxnet. The Stuxnet worm came to light in 2010 and aimed at industrial control systems controlling centrifuges for enriching uranium. The engine of Symantec worked out the Windows elements and the code that infected the hardware from Siemens that led to changes that were introduced inside the device without being shown to users (Falliere et al., 2011). The given research was conducted only because the authors were capable of reading the low machine code related to both platforms – PC and PLC.

WannaCry. In May 2017 'WannaCry' spread with the use of the EternalBlue technique to about 200,000 computers all over the world in the processes of hacking more than 150 countries. Checking one of the copies of the malware, the hacker found out that the program executed the code only is a special domain was not registered. As such, after it was registered in the domain, all copies trying to operate in the domain were turned off (Cloudflare, n.d.).

Mirai. In the 2016 Mirai denial-of-service attack, the botnet used devices such as routers, cameras and smart TVs whose processors operate using the MIPS and ARM architecture and an analysis of all the different binary codes that were compiled in order to understand how the Mirai botnet carried out the scanning and infecting of devices (Antonakakis, 2017). This case shows that as attacks are increasingly targeting embedded devices, forensic investigators will require knowledge of assembly programming beyond x86.

2.3.4 Digital Forensics and Incident Response

When conducting investigations, the investigators are more likely to retrieve the code from the memory rather than from the disk since advanced malware can operate without writing any codes on the disk. The memory forensic framework is able to retrieve the code that has been injected into the RAM so that the person conducting the forensic investigation can disassemble the codes and determine whether the malware is malicious or not (Ligh et al., 2014). Detection engineering requires the expert forensic analyst to have an understanding of how to read the low-level language.

2.4 Practical Reverse Engineering Demonstration

In order to apply these techniques, the SmartSignal executable that was generated earlier was treated just like an unknown file. The first step consisted of invoking the strip command on one

copy of the executable, since it is a common practice used by malware writers in order to establish the absence of function names.

Step 1: Identification. The readelf command was employed, as well as the file command, to reveal the fact that the sample is a 32-bit ELF executable for Intel 80386 architecture, statically linked and stripped. The entry point of the application is at address 0x8049000 (Figure 1). The nm command proves that there are no symbols left, and the names such as handle_ped cannot be used by the analyst anymore. Due to the stripping, the size of the file has decreased from 26,208 bytes to 16,968 bytes.

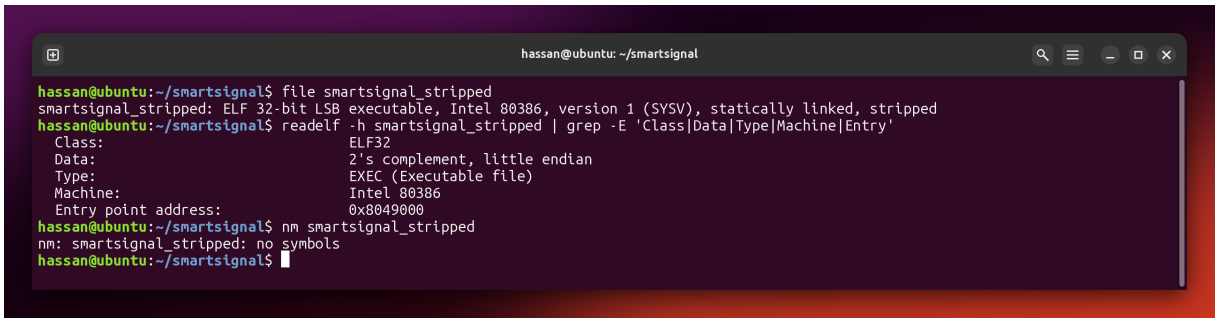


Figure 1: Identifying the stripped SmartSignal binary with file, readelf and nm

Step 2: String extraction. The strings utility extracts meaningful strings from the data segment of the sample shown in Figure 2. The strings section already makes clear the objective of the application being analyzed: it is clear that there are messages related to pedestrian requests from the system, various phases of the WALK system and laws concerning emergency vehicles. In many real-life cases of investigation strings contain URLs, file names, registry keys, ransom notes.

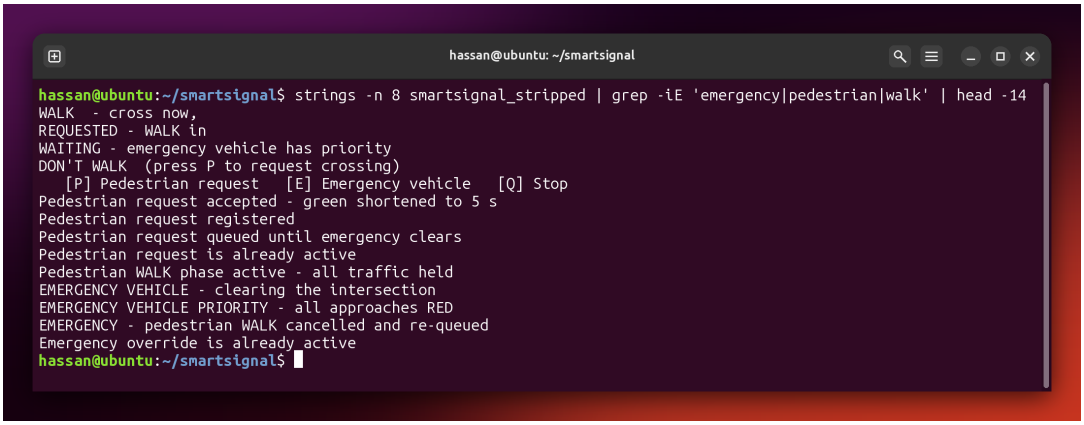


Figure 2: Strings extracted from the stripped binary reveal its behaviour

Step 3: Behaviour profiling from system calls. As a Linux program does not have any way to communicate with the operating system except via system calls, the analyst can analyze the program's capabilities by simply locating instances of int 0x80 instruction and values loaded

into EAX register just before the execution of the instruction itself. The disassembly was written to a file for further investigation (figure 3). Table 3 clarifies the findings for the analyst. Analysis of the disassembly showed that the program did not perform any kind of operations with networks, files or processes (e.g. no socketcall, open or execve calls were detected), which would be a great indication of possible illicit activity. Instead, the program was doing input, output, changing terminal settings, sleeping and polling.

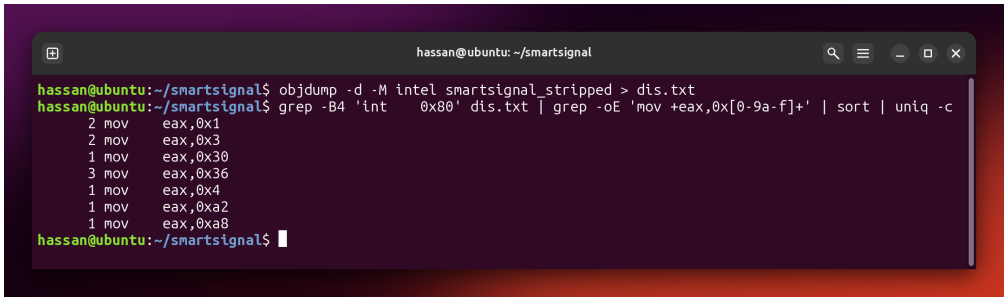


Figure 3: Counting system call numbers loaded into EAX before each int 0x80

Table 3: Behaviour inferred from the system call profile

EAX	System call	Count	Behaviour inferred
0x1	exit	2	Normal termination, including from a signal handler
0x3	read	2	Reads user input (line input and single keys)
0x4	write	1	Writes output to the terminal through a shared print routine
0x30	signal	1	Installs a signal handler (Ctrl+C)
0x36	ioctl	3	Gets and sets terminal attributes, which suggests raw keyboard mode
0xa2	nanosleep	1	Timed delays, consistent with a timer loop
0xa8	poll	1	Non-blocking check for available input

Step 4: Disassembly of specific logic. With the help of objdump, it is possible to disassemble the section of the code that receives the command to allow pedestrians to cross the street (depicted in Figure 4). The instruction that can be found at memory address of 0x8049fc0 runs the comparison between the data store at memory location 0x804c113 and the number 5: if the former is higher than the numeric value 5, 5 is assigned to it, and then the system obtains the address of the pedestrian instructions message. Thus, together with the similar strings, the rule "if more than 5 seconds of green light remain, cut the green to 5 seconds" becomes clear. This is how exactly the handle_ped function from the source code was reconstructed without having a direct access to it as well as decoding the machine code meaning of bytes 83 3D ... 05 and C7 05 ... 05, where the value 5 is encoded in the instructions.

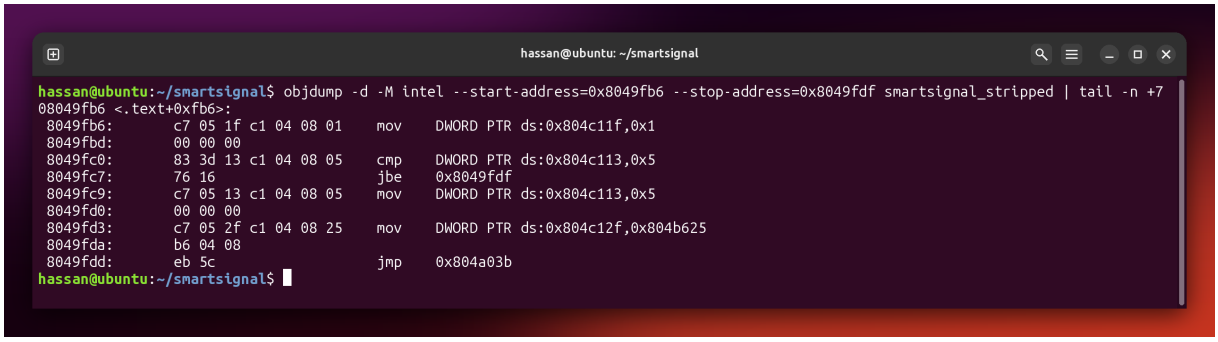


Figure 4: Disassembly of the pedestrian rule in the stripped binary

Step 5: Binary patching and tamper detection. Attackers and software crackers can alter programs simply by changing a couple of bytes in the software. For example, two bytes representing the integer 5 were altered to the integer 2. The cmp -l command displays which bytes are different, and disassembling the modified file shows that new assembly code has been utilized. The result of the modified version is that a user request causes the green light to last only 2 seconds while the words displayed on the screen still say "shortened to 5 s". The difference between what the program does in real life and the words printed on the screen is what investigators look for. One common way to show that a piece of software has been tampered with is to compare the target program with a good copy byte by byte or hash the target program and a known good copy of the software.

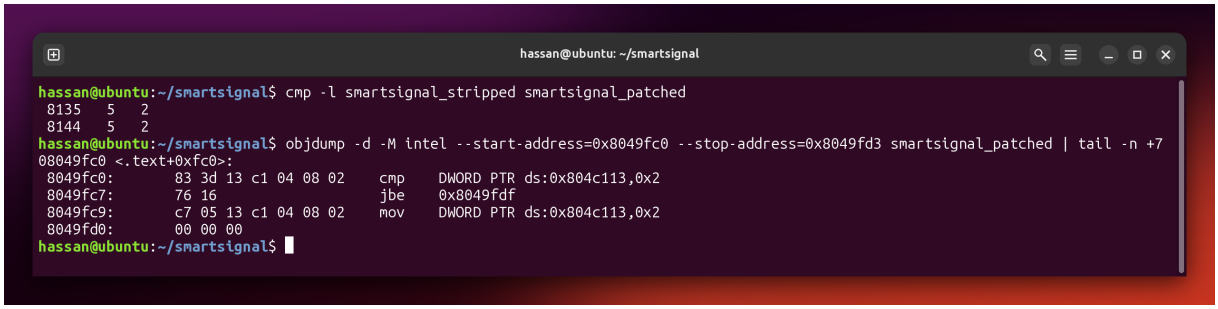


Figure 5: Locating and verifying the patched bytes

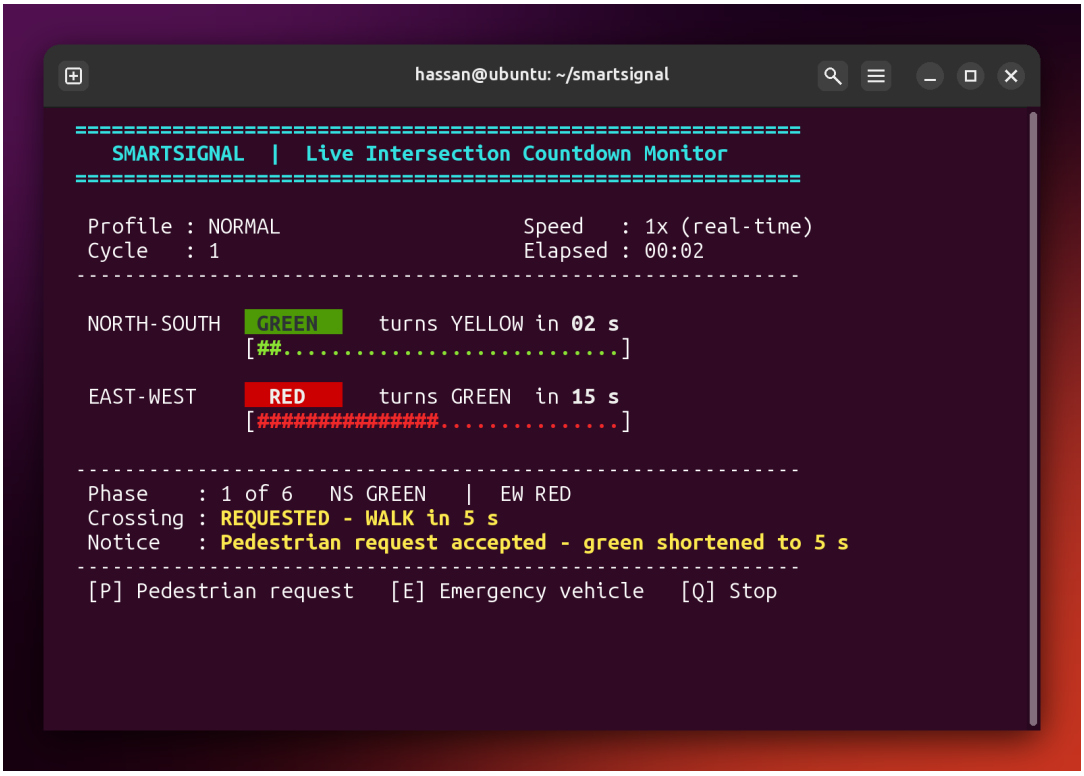


Figure 6: The patched binary cuts the green to 2 s while still displaying the original message

2.5 Critical Analysis

Table 4 summarises the strengths and limitations of assembly language identified in the research and experienced while developing SmartSignal.

Table 4: Strengths and limitations of assembly language

Aspect	Strengths	Limitations
Performance	Full control over the instructions executed; hand-tuned SIMD code can be many times faster for hot routines (Shilov, 2024)	Gains are limited to small routines; modern compilers are good enough for most whole programs
Hardware access	Direct access to registers, interrupts and privileged instructions needed by kernels and firmware	Code is tied to one architecture; x86 code does not run on ARM without emulation
Size and predictability	Small binaries and deterministic timing suit embedded and real-time systems	Development is slow; even printing a number needs a conversion loop and a system call
Security and forensics	Essential for malware analysis, reverse engineering and exploit analysis, because all software ultimately runs as machine code	Analysis is time-consuming; stripping, packing and obfuscation make it harder
Maintainability	Documents exactly what the processor does	Harder to read, debug and maintain than high-level code; mistakes such as an unbalanced stack can crash the program

It has been shown in existing research that the role of assembly language is changing rather than coming close to extinction as applications become less reliant on assembly. Nonetheless, minute as it may seem, assembly is becoming one of the fundamental skills of any analyst. The quick success of Ghidra (National Security Agency, 2023) as well as FFmpeg's vision of training new assembly programmers (FFmpeg, n.d.) indicates the sustained demand for assembly skills. In digital forensics, in particular, proliferation of IoT devices that run on ARM and MIPS processors suggests that analysts now need to know more than one instruction set (Antonakakis et al., 2017).

The development of SmartSignal underlines this point. While in high-level programming languages certain actions, for instance, the reading of a key before pressing the ENTER button, may seem simple, in Linux in order to perform this kind of operation one needs to acquire some basic knowledge of the Linux terminal and of some system calls. Nevertheless, the final outcome is an executable of less than 17 KB which is however a "known" program and, as is shown in Section 2.4, whose behavior can be completely and unambiguously reconstructed from its disassembly.

3.0 System Design

3.1 System Functionalities

Table 5 lists the functions of SmartSignal and the main assembly constructs each one uses.

Table 5: SmartSignal functionalities

No.	Function	Description	Key constructs
1	Live intersection simulation	Six-phase signal cycle with per-second countdowns, countdown bars, phase, crossing and notice lines	Timing loop, DEC/JNZ, SUB, table lookup
2	Time-to-change projection	Calculates the seconds until each approach's light changes, including queued walk or emergency time	LOOP, ADD, CMP/JNE
3	Pedestrian request (P)	Cuts a long green to 5 s and adds a WALK interval to the all-red phase	CMP/JBE, ADD
4	Emergency override (E)	Ends the green immediately, then holds all approaches red for an extra 8 s	CMP/JE branching, ADD
5	Night mode	Flashing amber on all approaches for 30 s or until Q is pressed	LOOP, XOR toggle, SHR
6	Traffic profiles	Loads Normal, Peak Hour, Off-Peak or School Zone timings from a table	SUB (ASCII to index), table addressing
7	Configure timings	Table-driven settings menu with range-checked numeric input	Table loop, ADD, CMP/JL/JG
8	Timing plan	Phase table, cycle length, red times, cycles per hour and green share	LOOP with ADD, SUB, MUL/DIV
9	Session statistics	Totals and averages across all simulations in the session	ADD accumulation, DIV
10	Safe exit and Ctrl+C	Restores the terminal settings and cursor on every exit path	Signal handler, JMP

3.2 Signal Phase Model

SmartSignal treats the intersection as a finite-state device that consists of six phases of operation according to the data included in the Table 6 and the data included in the Figure 7. Each road gets green light, followed by yellow and followed by all-red phase during which all roads are stopped. The specified sequence duplicates the principle of changing and clearing intervals in actual traffic light operation (National Academies of Sciences, Engineering, and Medicine, 2015). Times required for every phase are introduced in a six-entry record table formed as a result of adjustment process each time a setting is changed. Besides the database including six entries, two remaining six-byte databases contain data on the basis of which the character of light being on.

Table 6: The six signal phases (durations shown for the Normal profile)

Phase	North-South	East-West	Duration
1	GREEN	RED	NS green time (15 s)
2	YELLOW	RED	Amber time (3 s)
3	RED	RED	All-red clearance (2 s), plus walk or emergency hold if queued
4	RED	GREEN	EW green time (10 s)
5	RED	YELLOW	Amber time (3 s)
6	RED	RED	All-red clearance (2 s), plus walk or emergency hold if queued

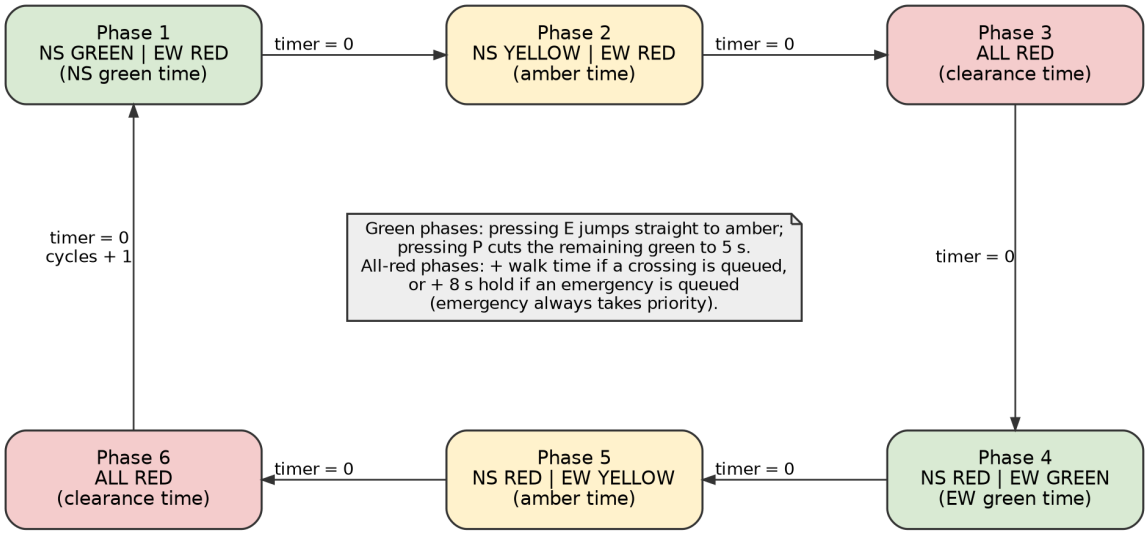


Figure 7: Signal phase state diagram

Once the time set for a phase elapses, the next phase commences; after phase 6 has been reached, it reverts to phase 1 and cycles the count. Pedestrian and emergency events interfere with this process. A pedestrian call reduces the green phase remaining for more than 5 seconds to 5 seconds, while the following yellow phase continues for the duration of the pedestrian crossing. An emergency request interrupts the green and goes straight to amber, having an additional amber duration of 8 seconds added.

3.3 Main Program Flowchart

The program flow is illustrated in Figure 8. The first thing that the program does after it is started is to instantiate a Ctrl+C handler and to create the phase duration schedule. Following this, a main menu is shown indicating the profile being used and the current cycle length information. The input is expected to be a single character, and any incorrect input results in

displaying an error message and going back to the main menu. Once the user makes a valid input, the execution jumps to the corresponding module, after which it returns back to the main menu. Selecting 0 returns the terminal back to the default state and finishes the program.

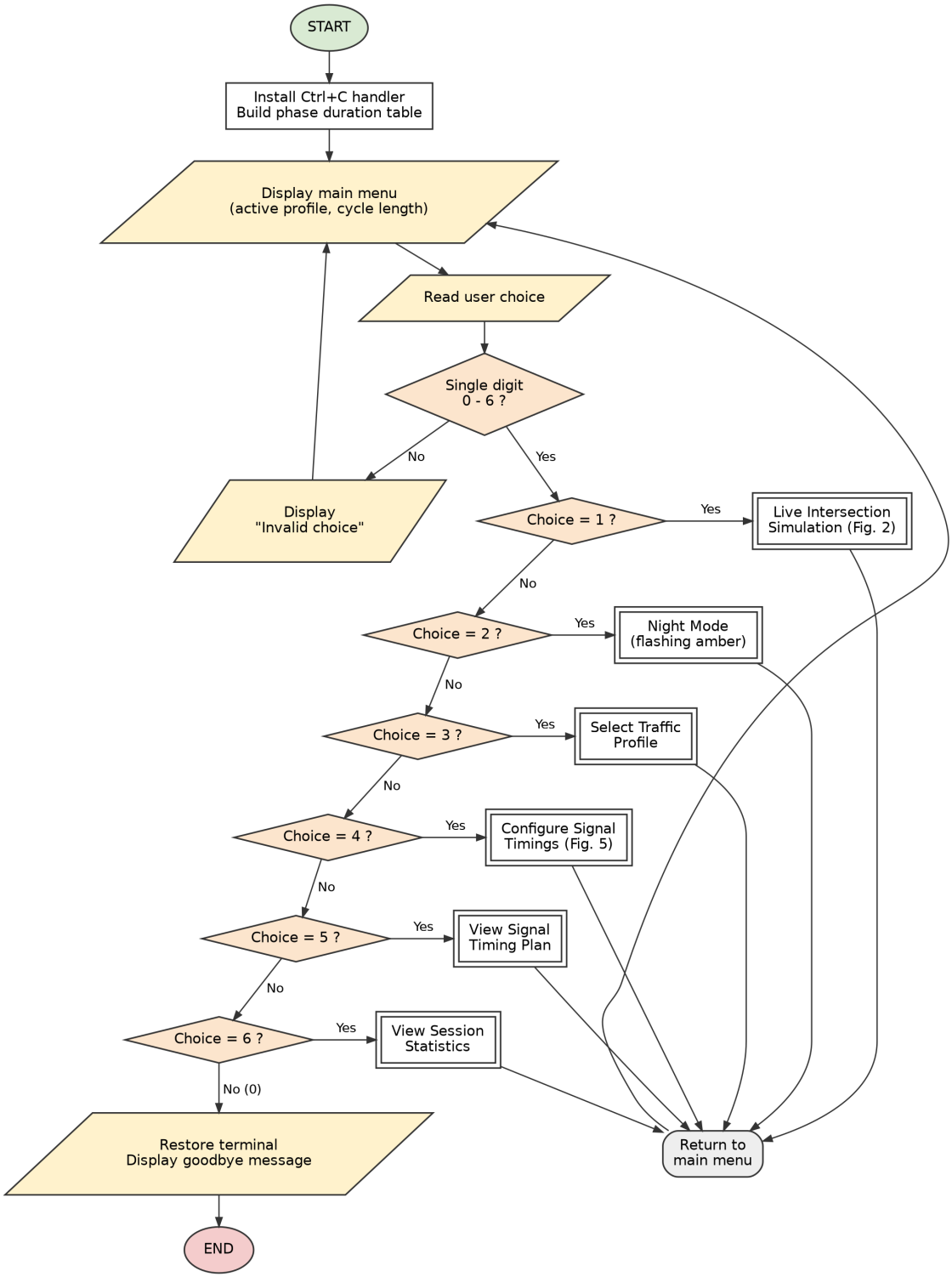


Figure 8: Main program flowchart

3.4 Live Simulation Flowchart

The live simulation (Figure 9) is the heart of the entire system. Each second that is simulated is divided into ten slices. Within each slice, the program waits for 100 milliseconds (or 20 milliseconds in fast mode) and checks for key-presses in a non-blocking fashion. When the user presses P or E, the appropriate handler is invoked and the dashboard is refreshed immediately. The simulation terminates when the user presses Q. Once ten slices have passed, the time elapsed is increased and the green-time statistics are updated.

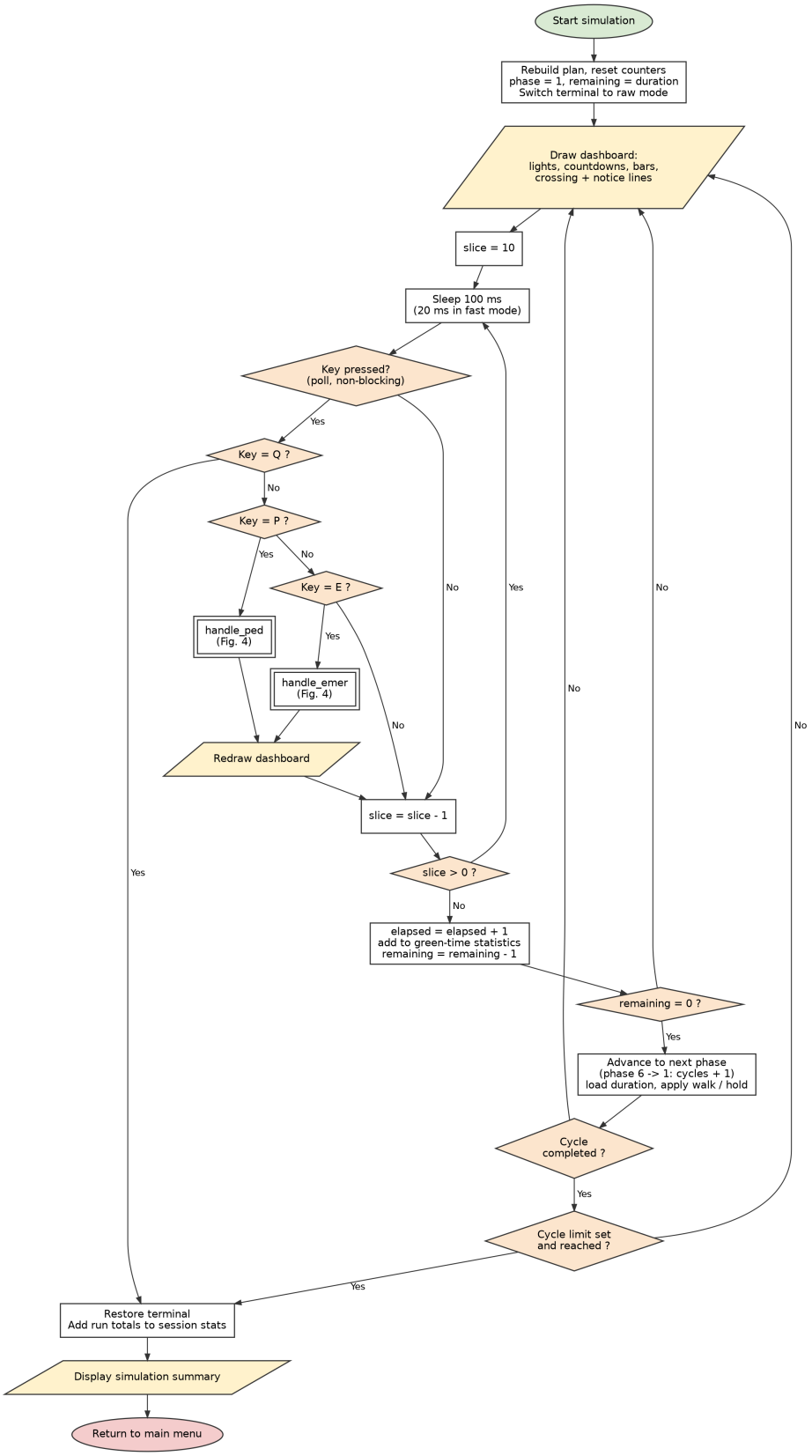


Figure 9: Live simulation flowchart

3.5 Pedestrian and Emergency Event Handling

The role of these two event handlers is explained in Figure 10. Both of them reject the same request if it comes in its duplicate form. The pedestrian handler recognizes three possible cases. While the all-red signal is in effect a walk is allowed from the very beginning but only if there is no emergency hold active. During green the request is queued until it is realized that the green phase can last for only 5 seconds. If amber is on, the request will be queued until the next all-red signal appears. The other event handler (the emergency handler) simply increments the override counter and can start either a jump from green to amber, a pause during the current amber, or returns to the all-red signal if this is already active.



Figure 10: Pedestrian request and emergency override flowcharts

3.6 Configuration and Input Validation Flowchart

The settings menu is driven by a table (see Figure 11). Each setting is stored in a table that contains seven entries. For each setting, the table keeps a pointer to the relevant label, the pointer to the variable, the minimum and maximum limits, the unit string and a flag that shows that the profile is CUSTOM while a timing value is changed. The table allows the programmer to display each row in the loop. The variable is validated using the limits stored in the table, which means that new settings can be added to the program without the need to write new menu code.

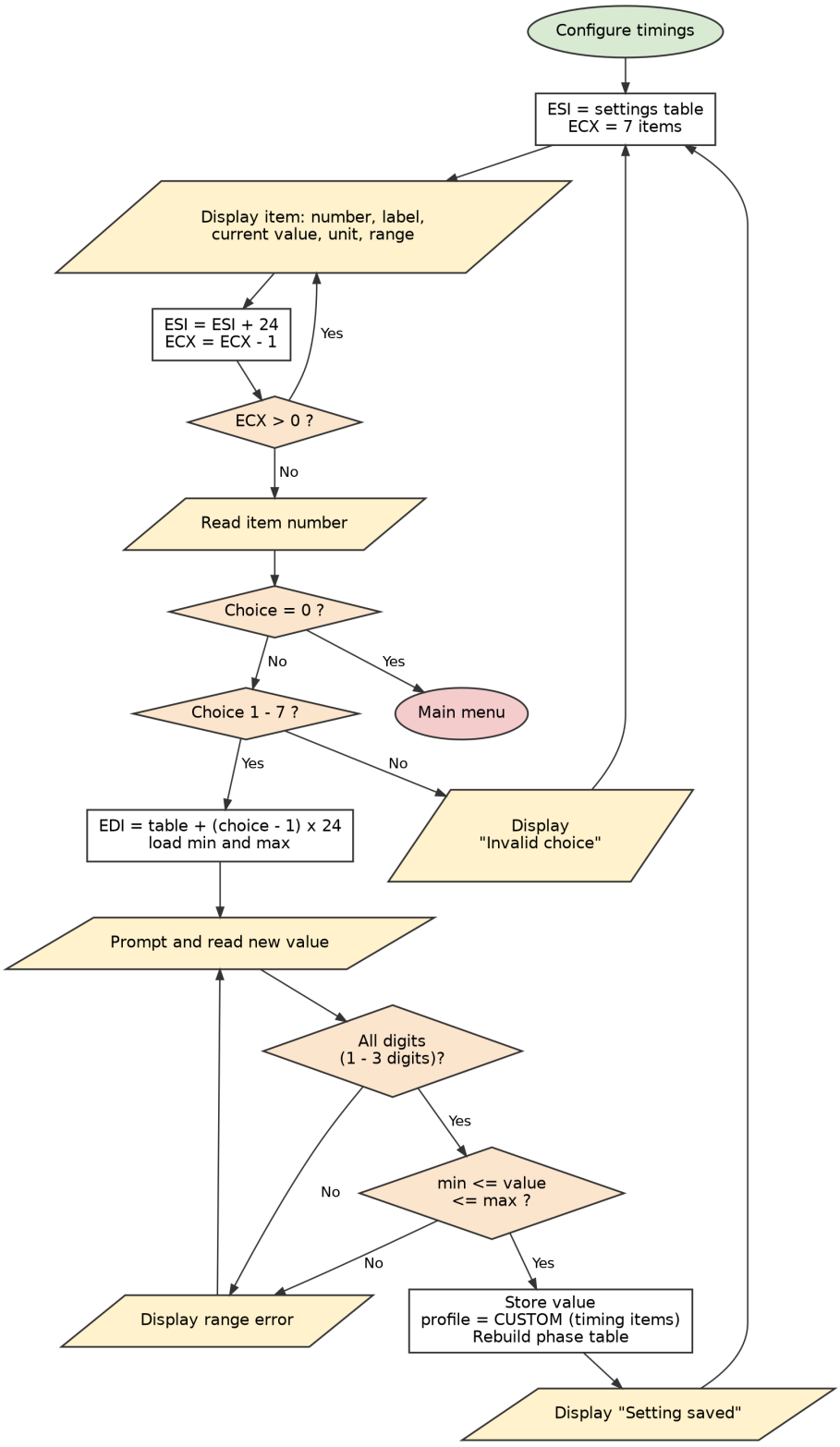


Figure 11: Configure signal timings flowchart

3.7 Data Design

Table 7 describes the main data structures stored in the `.data` and `.bss` sections.

Table 7: Main data structures

Data item	Type and size	Purpose
phase_dur	6 doublewords	Duration of each phase, rebuilt from the settings
ns_col, ew_col	6 bytes each	Light colour of each approach per phase (0 red, 1 yellow, 2 green)
phase_txt	6 pointers	Description string for each phase
prof_tab	4 entries x 24 bytes	NS green, EW green, amber, all-red and walk times, and name pointer of each profile
cfg_tab	7 entries x 24 bytes	Label, variable pointer, minimum, maximum, unit and custom flag of each setting
phase, remaining, cycles, elapsed	Doublewords	Current state of the running simulation
ped_pending, walk_active, emer_pending, emer_active	Doublewords (flags)	Pedestrian and emergency state
st_runs to st_night	8 doublewords	Session statistics accumulated across simulations
term_orig, term_raw	64-byte buffers	Saved and modified terminal settings (termios structure)

4.0 System Implementation

This section explains the most important parts of the source code. The complete program, `smartsignal.asm` (about 1,700 lines including comments), is submitted as a separate file.

4.1 Program Structure and Modular Design

The program is composed of several routines, with each of them having a specific role and a comment that helps to understand what the inputs and outputs are. Each of the routines saves the values of registers that have no other uses because they are handled, traditionally, by either using PUSHAD/POPAD method or via ordinary PUSH/POP pairs. The code readability is supported by two macros: PRINT, which is used for printing a string while preserving the values of all registers, and SET_NOTICE responsible for setting a message on the display. Furthermore, named constants with the help of the %define instruction are used instead of numbers until their meanings are made clear, e.g. PH_EW_GREEN for 3 or EMER_HOLD for 8. The core routines are summarized in Table 8.

Table 8: Main routines grouped by responsibility

Group	Routines	Responsibility
Screens	main_menu, run_simulation, night_mode, profile_menu, config_menu, timing_plan, show_stats, exit_program	One routine per menu option
Signal logic	build_plan, cycle_length, enter_phase, advance_phase, time_to_change, secs_until_walk, handle_ped, handle_emer	Phase model, countdown projection and event rules
Display	draw_dashboard, draw_road, build_bar, print_colour_word, print_header	Live dashboard and formatted output
Terminal and timing	term_raw_on, term_restore, check_key, subtick_delay, sleep_ms, sigint_handler	Keyboard mode, non-blocking input, delays and Ctrl+C
Input	read_line, parse_int, read_int_range, wait_enter	Reading and validating user input
Output	print_str, print_num, print_num2, print_pad2, print_mmss	Converting numbers to text and writing to the screen

4.2 Arithmetic Operations (ADD and SUB)

The program makes good use of ADD and SUB when performing timing calculations, projections, statistics and conversions. Countdown itself is a subtraction done once for every

simulated second. DEC takes the best of SUB 1, it sets zero flag thus affecting various tests by JNZ.

```
.countdown:
        dec     dword [remaining]       ; countdown (SUB 1)
        jnz     .tick                   ; time left -> keep counting
```

The timing plan calculates the red time of each approach by subtracting its green and amber times from the total cycle length held in EDI:

```
        ; red time = cycle - green - amber   (SUB)
        PRINT   t_nsred
        mov     eax, edi
        sub     eax, [ns_green]
        sub     eax, [amber_t]
        mov     ebp, eax                ; keep NS red time
```

Upon user input, parse_int executes the process of digit conversions using SUB and builds the number from previously calculated total with multiplication involving only shifting and adding operations.

```
        sub     bl, '0'                 ; ASCII -> value (SUB)
        mov     edx, eax                ; EAX * 10 = EAX*8 + EAX*2
        add     eax, eax
        shl     edx, 3
        add     eax, edx
        add     eax, ebx                ; append the new digit (ADD)
```

ADD is also used to accumulate session statistics after each simulation (`add [st_ped], eax`), to extend all-red phases by the walk or emergency time, and to convert upper-case key presses to lower case (`add al, 'a' - 'A'`).

4.3 Loops

The program uses both the LOOP instruction, which decrements ECX and jumps while ECX is not zero, and explicit counter loops built from DEC and JNZ. `cycle_length` uses LOOP to add up the six phase durations:

```
; cycle_length: EAX = sum of all phase durations (LOOP + ADD)
cycle_length:
        push    ecx
        push    esi
        xor     eax, eax
        mov     esi, phase_dur
        mov     ecx, NUM_PHASES
.sum:
        add     eax, [esi]
        add     esi, 4
        loop    .sum
        pop     esi
        pop     ecx
        ret
```

`build_bar` draws the countdown bar by looping 30 times, writing '#' while the position is below the remaining seconds and '.' afterwards:

```
        mov     edi, barbuf
        xor     ebx, ebx                ; position counter
        mov     ecx, BAR_MAX
.fill:
        mov     byte [edi], '.'
        cmp     ebx, eax
        jae     .next
        mov     byte [edi], '#'
.next:
        inc     edi
        inc     ebx
        loop    .fill
        mov     byte [edi], 0
```

The simulation's primary timing loop consists of a nested loop. The outer loop iterates once per simulated second, while the inner runs 10 times, pausing and checking for input each time. Since the body of an inner loop extends past the 127-byte limitation of the LOOP instruction, it employs DEC ECX and JNZ instead:

```
.tick:                                  ; ---- one simulated second ----
        call    draw_dashboard
        mov     ecx, SUBTICKS
.subtick:                               ; split second into 10 slices
        call    subtick_delay
        call    check_key
        cmp     al, 'q'
        je      .user_stop
        cmp     al, 'p'
        jne     .not_ped
        call    handle_ped
        call    draw_dashboard
        jmp     .next_sub
.not_ped:
        cmp     al, 'e'
        jne     .next_sub
        call    handle_emer
        call    draw_dashboard
.next_sub:
        dec     ecx
        jnz     .subtick
```

4.4 Conditional and Unconditional Jumps

Every decision in the program is accomplished using conditional jumps. The main screen compares the input to each option and, using JE, jumps to the corresponding module. All other

input is directed to the failure message and is followed by an unconditional jump to main_menu:

```
        call    read_line
        cmp     ecx, 1                  ; exactly one character expected
        jne     .invalid
        mov     al, [inbuf]
        cmp     al, '1'
        je      run_simulation
        cmp     al, '2'
        je      night_mode
        cmp     al, '3'
        je      profile_menu
        cmp     al, '4'
        je      config_menu
        cmp     al, '5'
        je      timing_plan
        cmp     al, '6'
        je      show_stats
        cmp     al, '0'
        je      exit_program
.invalid:
        PRINT   fg_red
        PRINT   err_choice
        PRINT   rst
        call    wait_enter
        jmp     main_menu
```

The pedestrian handler shows how equality tests (JE, JNE) and an unsigned comparison (JBE) combine to implement the request rules described in Section 3.5:

```
; handle_ped: processes a pedestrian crossing request (key P)
handle_ped:
        pushad
        cmp     dword [walk_active], 0
        jne     .already
        cmp     dword [ped_pending], 0
        jne     .already
        mov     ebx, [phase]
        call    is_allred
        je      .in_allred
        cmp     ebx, PH_NS_GREEN
        je      .green
        cmp     ebx, PH_EW_GREEN
        je      .green
        mov     dword [ped_pending], 1  ; yellow: wait for all-red
        SET_NOTICE msg_ped_reg
        jmp     .done
.green:
        mov     dword [ped_pending], 1
        cmp     dword [remaining], PED_SHORTEN
        jbe     .registered
        mov     dword [remaining], PED_SHORTEN   ; shorten the green
        SET_NOTICE msg_ped_short
        jmp     .done
```

Signed comparisons (JL and JG) are used in `read_int_range` to enforce the minimum and maximum of a setting, and the carry flag returned by `parse_int` is tested with JC to detect invalid characters.

4.5 Real-Time Countdown Projection

The time_to_change function represents the same idea as the "green in 12 sec" display used in navigation apps. It begins from the time left in the current phase and goes through the next phases while adding the respective durations until the color of the various phases stays unchanged. In addition, if there is a pedestrian phase or an emergency stop in the queue for the next all-red phase, the time period will be added and, accordingly, the countdown will be accurate after the event.

```
; time_to_change: EAX = seconds until the light of one approach changes
; Input : EDI -> colour table of that approach (ns_col or ew_col)
; Adds the remaining time of the current phase to the durations of the
; following phases that keep the same colour, including any projected
; pedestrian walk or emergency hold in the next all-red phase.
time_to_change:
        push    ebx
        push    ecx
        push    edx
        push    esi
        mov     ebx, [phase]
        mov     dl, [edi + ebx]         ; current colour of this approach
        mov     eax, [remaining]
        mov     ecx, NUM_PHASES - 1     ; look at most 5 phases ahead
        xor     esi, esi                ; 1 = next all-red already counted
.next:
        inc     ebx
        cmp     ebx, NUM_PHASES
        jb      .check
        xor     ebx, ebx                ; wrap to phase 1
.check:
        cmp     [edi + ebx], dl
        jne     .done                   ; colour changes -> stop summing
        add     eax, [phase_dur + ebx*4]
        call    is_allred
        jne     .cont
        test    esi, esi
        jnz     .cont
        mov     esi, 1
        cmp     dword [emer_pending], 0
        je      .ped
        add     eax, EMER_HOLD
        jmp     .cont
.ped:
        cmp     dword [ped_pending], 0
        je      .cont
        add     eax, [walk_t]
.cont:
        loop    .next
.done:
        pop     esi
        pop     edx
        pop     ecx
        pop     ebx
        ret
```

The same routine serves both approaches because the caller passes the address of the approach's colour table in EDI, which avoids duplicating code.

4.6 Non-Blocking Keyboard Input

A normal terminal delivers input only after ENTER is pressed and echoes every key, which would freeze the countdown while the program waits. `term_raw_on` reads the current settings with `ioctl(TCGETS)`, copies them, clears the ICANON and ECHO flags, sets VMIN and VTIME to zero, and applies the new settings with `ioctl(TCSETS)` (Kerrisk, 2010):

```
; term_raw_on: switches the terminal to non-canonical, no-echo mode so
; single key presses can be read while the countdown keeps running
term_raw_on:
        pushad
        mov     eax, SYS_IOCTL
        mov     ebx, STDIN
        mov     ecx, TCGETS
        mov     edx, term_orig
        int     0x80
        test    eax, eax
        js      .done                   ; not a terminal -> skip
        cld
        mov     esi, term_orig
        mov     edi, term_raw
        mov     ecx, 64
        rep     movsb
        and     dword [term_raw + LFLAG_OFS], ~(ICANON | ECHO)
        mov     byte [term_raw + VMIN_OFS], 0
        mov     byte [term_raw + VTIME_OFS], 0
        mov     eax, SYS_IOCTL
        mov     ebx, STDIN
        mov     ecx, TCSETS
        mov     edx, term_raw
        int     0x80
        mov     dword [raw_on], 1
```

`check_key` then uses the `poll` system call with a timeout of zero to test whether a key is waiting before reading it, so the countdown never blocks:

```
        mov     dword [pfd], STDIN
        mov     word [pfd + 4], POLLIN
        mov     word [pfd + 6], 0
        mov     eax, SYS_POLL
        mov     ebx, pfd
        mov     ecx, 1
        xor     edx, edx                ; timeout 0 = do not wait
        int     0x80
        cmp     eax, 1
        jl      .none
        test    word [pfd + 6], POLLIN
        jz      .none
        mov     eax, SYS_READ
        mov     ebx, STDIN
        mov     ecx, keybuf
        mov     edx, 1
        int     0x80
```

The original settings are restored by `term_restore` whenever a simulation or night mode ends, when the program exits and in the Ctrl+C handler.

4.7 Input Validation and Robustness

`read_int_range` repeats the prompt until `parse_int` succeeds and the value lies between the minimum and maximum taken from the settings table:

```
; read_int_range: prompts (ESI) until a number between EBX and EDX is
; entered. Returns the value in EAX.
read_int_range:
.ask:
        call    print_str
        call    read_line
        call    parse_int
        jc      .bad
        cmp     eax, ebx
        jl      .bad
        cmp     eax, edx
        jg      .bad
        ret
```

The read_line method provides input one byte at a time, ignores carriage returns, limits the input to a maximum of 30 characters, and exits cleanly even if the input stream is closed, thereby preventing it from hanging infinitely in case of an error. In addition, a signal handler is set up during the start of the program which makes sure that pressing Ctrl+C while the program is running restores the terminal echo and cursor control:

```
_start:
        ; Install a Ctrl+C handler so the terminal is always restored
        mov     eax, SYS_SIGNAL
        mov     ebx, SIGINT
        mov     ecx, sigint_handler
        int     0x80

; sigint_handler: restores the terminal when Ctrl+C is pressed
sigint_handler:
        call    term_restore
        PRINT   rst
        PRINT   show_cur
        PRINT   intr_msg
        mov     eax, SYS_EXIT
        xor     ebx, ebx
        int     0x80
```

4.8 Testing

The program compiled successfully with no errors or warnings, even with all possible NASM warnings turned on (-w+all). The code was tested in an automated terminal session where real keystrokes were sent and following screens recorded, as well as by means of QEMU i386 emulator that runs it in an ARM-based Ubuntu virtual machine.

Table 9: Test cases and results

ID	Test case	Input	Expected result	Actual result	Status
T01	Main menu display	Start program	Seven options, profile NORMAL, cycle 35 s	As expected (Figure 12)	Pass

ID	Test case	Input	Expected result	Actual result	Status
T02	Invalid menu choice	8, 9 or 12	Error message, menu shown again	As expected (Figure 29)	Pass
T03	Live countdown	Option 1	After 2 s: NS green 13 s left; EW green in 18 s	As expected (Figure 13)	Pass
T04	Pedestrian request during long green	P after 2 s	Green cut to 5 s; WALK countdown = green left + amber	As expected (Figure 15)	Pass
T05	Pedestrian request during amber	P in amber (Off-Peak)	Request queued; EW countdown includes 6 s walk	EW green in 9 s (2 + 1 + 6)	Pass
T06	Duplicate request	P pressed twice	"Already active" notice, no extra time added	As expected	Pass
T07	WALK phase	Wait after T04	All-red extended by walk time with WALK countdown	As expected (Figure 16)	Pass
T08	Emergency during green	E in green	Jump to amber, then all-red with 8 s hold	As expected (Figure 17, Figure 18)	Pass
T09	Emergency during WALK	E during walk	Walk cancelled and re-queued, hold restarted	As expected	Pass
T10	Cycle limit	Limit 2, fast mode	Stops after 2 cycles: "cycle limit reached"	As expected (Figure 20)	Pass
T11	Invalid timing input	abc, 99, 70, x5	Range error repeated until a valid value	As expected (Figure 25)	Pass
T12	Profile selection	3, then 2	PEAK HOUR loaded, cycle 54 s	As expected (Figure 24)	Pass
T13	Timing plan arithmetic	Peak Hour profile	Cycle 54 s, NS red 20 s, EW red 38 s, 66 cycles/h, 55 % and 22 %	Matches manual calculation	Pass
T14	Night mode	2, then Q	Amber flashes every 0.5 s; Q exits	As expected (Figure 21, Figure 22)	Pass
T15	Statistics	6 after two runs	Average cycle = time / cycles	94 s / 2 = 47 s	Pass
T16	Ctrl+C during simulation	Ctrl+C	Terminal and cursor restored, clean exit	As expected	Pass
T17	Exit	0	Goodbye message, exit status 0	As expected (Figure 30)	Pass
T18	Closed input stream	Run with input from /dev/null	Program ends cleanly with no endless loop	Exit status 0	Pass

5.0 User Manual

5.1 System Requirements

- A computer running Linux (Ubuntu 22.04 or later recommended), either natively or in a virtual machine.
- NASM 2.15 or later and the GNU linker (ld).
- A terminal window of at least 80 x 26 characters with ANSI colour support. The default Ubuntu Terminal works.
- On ARM-based computers such as Apple Silicon Macs (M1 to M4): the QEMU user-mode emulator, as described in Section 5.2.

5.2 Installation

Option A: Ubuntu on an Intel or AMD (x86) computer. Open a terminal and install the assembler and linker:

```
sudo apt update
sudo apt install nasm binutils
```

Option B: Ubuntu ARM virtual machine on an Apple Silicon Mac (for example in UTM, Parallels or VMware Fusion). The processor is ARM, so the 32-bit x86 program is linked with an x86 cross-linker and run through QEMU:

```
sudo apt update
sudo apt install nasm binutils-x86-64-linux-gnu qemu-user
```

Copy `smartsignal.asm` into a folder, for example `~/smartsignal`, and open a terminal in that folder.

5.3 Building and Running

On an x86 computer:

```
nasm -f elf32 smartsignal.asm -o smartsignal.o
ld -m elf_i386 smartsignal.o -o smartsignal
./smartsignal
```

On an ARM virtual machine (Apple Silicon):

```
nasm -f elf32 smartsignal.asm -o smartsignal.o
x86_64-linux-gnu-ld -m elf_i386 smartsignal.o -o smartsignal
qemu-i386 ./smartsignal
```

The first command assembles the source into a 32-bit object file, the second links it into an executable, and the third starts the program. For the best display, maximise the terminal window before starting.

5.4 Using the Main Menu

The main menu (Figure 12) lists seven options and shows the active traffic profile and the current cycle length. Type the number of an option and press ENTER. Any other input displays an error message (Figure 29); press ENTER to return to the menu.

5.5 Running a Live Simulation

Choose option 1 to start the simulation (Figure 13). The dashboard updates every second and shows:

- **Profile, Speed, Cycle and Elapsed:** the active profile, real-time or fast mode, the current cycle number (and the limit, if one is set) and the simulated time.
- **North-South and East-West rows:** the current light of each approach, the seconds until it changes (for example "turns GREEN in 18 s") and a countdown bar with one block per remaining second.
- **Phase:** the current phase number (1 to 6) and which approaches are moving.
- **Crossing:** DON'T WALK, the time until WALK after a request, or the WALK countdown while pedestrians cross.
- **Notice:** the latest event, such as a shortened green or an emergency override.

Keys work immediately, without pressing ENTER (Table 10).

Table 10: Live simulation controls

Key	Action
P	Request a pedestrian crossing. A green with more than 5 s left is cut to 5 s, and WALK runs during the next all-red phase (Figure 15, Figure 16).
E	Emergency vehicle override. The current green ends, amber runs, then all approaches are held red for an extra 8 s (Figure 17, Figure 18).
Q	Stop the simulation and show the summary (Figure 19).
Ctrl+C	Emergency exit from the program. The terminal settings are restored automatically.

If a cycle limit is set (Section 5.8), the simulation stops automatically after that number of cycles (Figure 20). Press ENTER on the summary screen to return to the main menu.

5.6 Night Mode

Select option 2 to start night mode operation. Both methods are displayed with an amber light once in every half second. Time remaining until returning to the selection menu is shown on the timer, while Q can exit early (Figures 21,22).

5.7 Selecting a Traffic Profile

Choose option 3 for viewing available profiles and their times (Figure 23). To enable the profile simply enter it with the number between 1-4 or enter 0 for returning to the main menu. Confirmation about loading will appear on the display (Figure 24) and this profile will be used afterwards.

5.8 Configuring Signal Timings

Choose option 4 for displaying and modifying settings (Figure 26). Simply enter required setting number plus ENTER. Then enter new value. It would reject inputs which are out of range and contain letters with that information displayed on the screen (Figure 25). Changing the timer would change Profile to CUSTOM.

There are two settings that can be demonstrated:

- Simulation speed mode: 1 means normal speed; 2 means five-fold speed (Figure 14).
- Cycle limit: 0 means to run until Q is pressed and numbers 1 through 99 to run for the corresponding number of cycles.

5.9 Viewing the Signal Timing Plan

Select option 5 to view the phase table of the present settings along with the computed cycle time, the cycle red time for every approach, the longest waiting time for a green signal, the number of cycles running per hour and the percentage of green time for each of the approach (Figure 27)

5.10 Viewing Session Statistics

Select option 6 to view the statistics for all the simulations performed since the beginning of the program: the number of runs made, the total simulation time, the total number of cycles completed, the average cycle time, the green time by approach with its percentage, pedestrian crossings performed, emergency interventions, and night-mode operations (Figure 28).

5.11 Exiting the Program

Choose option 0 to exit. The terminal settings are restored and a goodbye message is displayed (Figure 30).

5.12 Troubleshooting

Table 11: Common problems and solutions

Problem	Cause	Solution
"Exec format error" when running ./smartsignal	A 32-bit x86 program is being run on an ARM computer	Install qemu-user and run qemu-i386 ./smartsignal
"i386 architecture of input file is incompatible"	The linker was run without the 32-bit option	Add -m elf_i386 to the ld command
Strange characters such as [1;32m appear	The terminal does not support ANSI colours	Use the default Ubuntu Terminal
Typed text is invisible after a forced shutdown	The terminal was left in raw mode	Type reset and press ENTER
Lines wrap or the dashboard overlaps	The terminal window is too small	Enlarge the window to at least 80 x 26 characters
Countdown is too slow for a demonstration	Real-time mode is active	Set the simulation speed mode to 2 in option 4

6.0 System Screenshots

The following screenshots were captured from the running program and show each feature of the system.

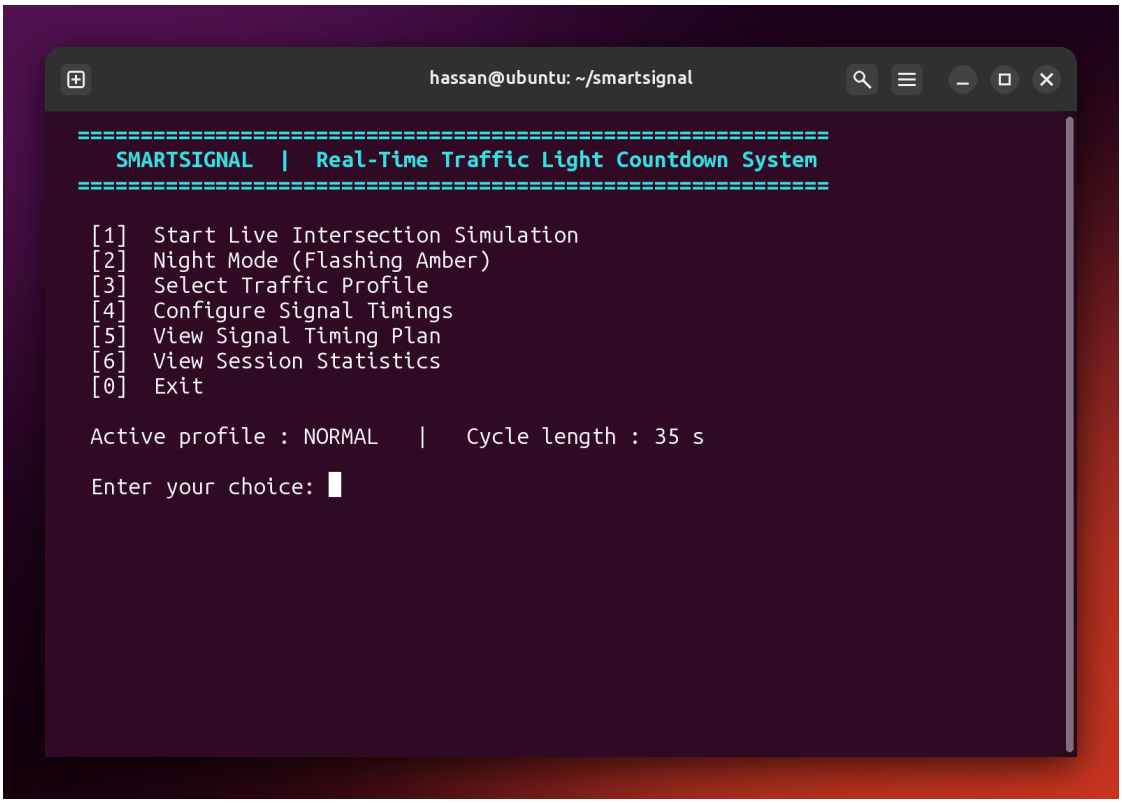


Figure 12: Main menu

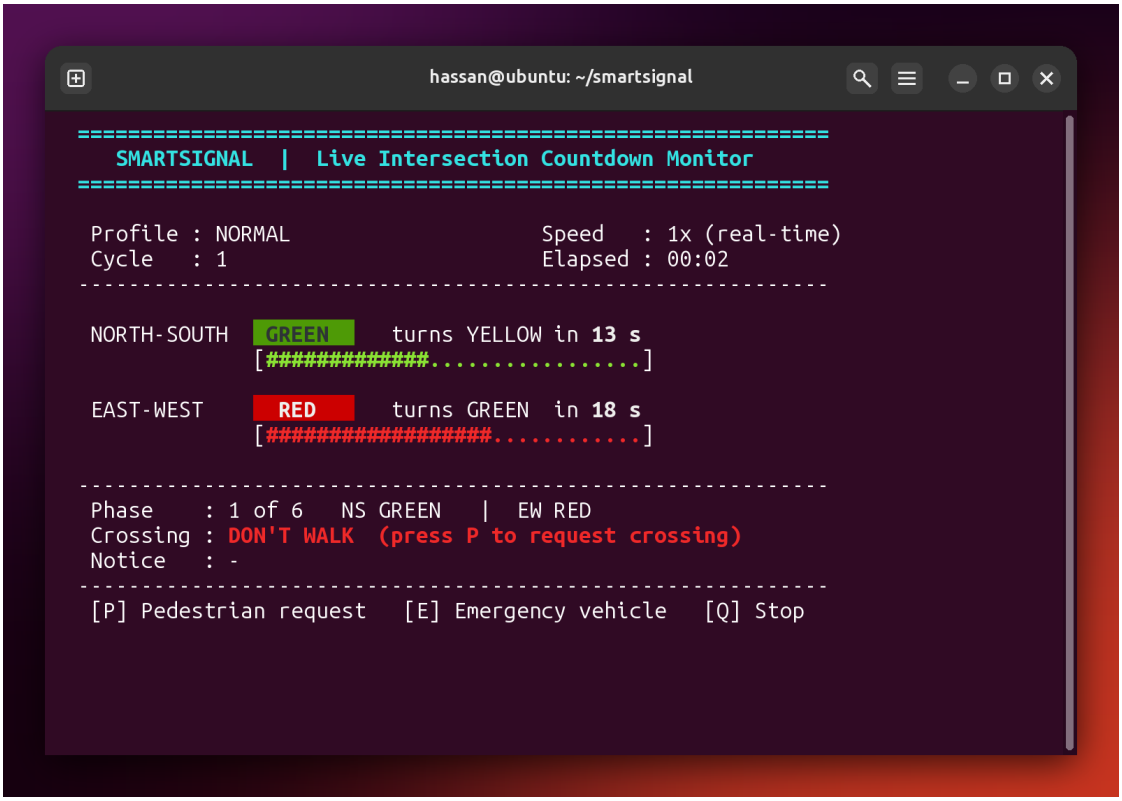


Figure 13: Live simulation: North-South green with countdowns for both approaches

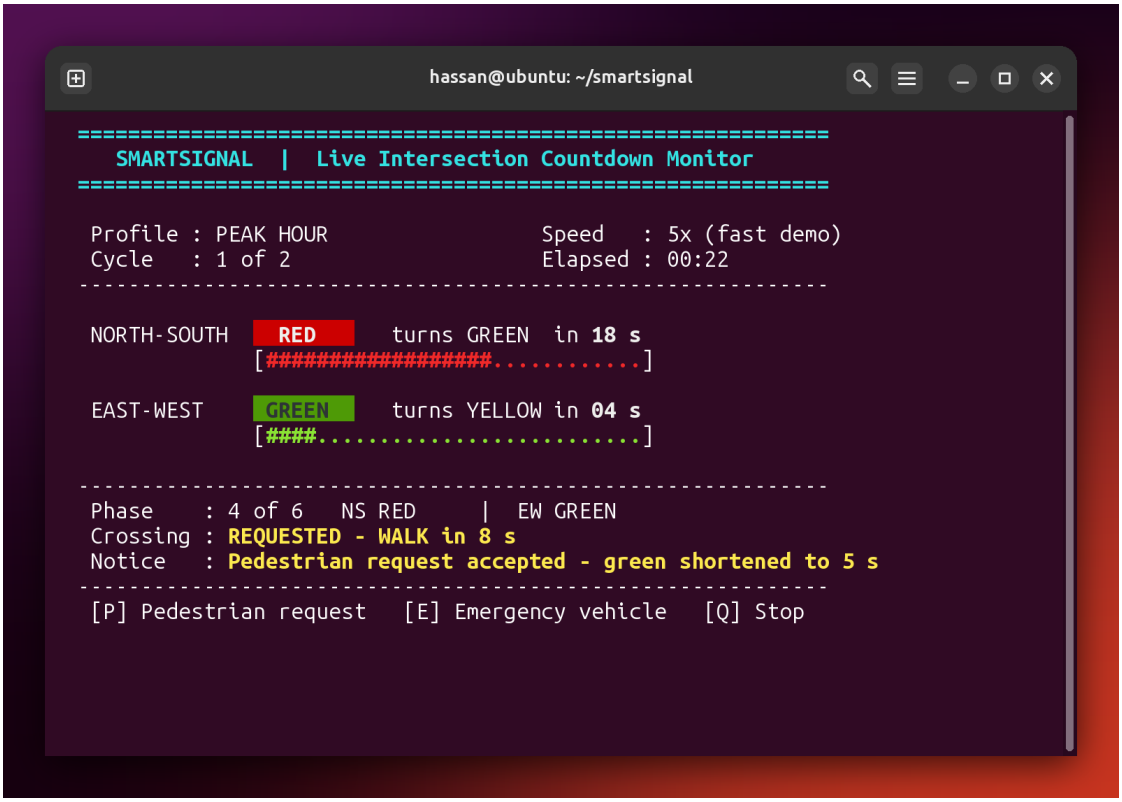


Figure 14: Simulation in fast mode (5x) with the Peak Hour profile and a cycle limit

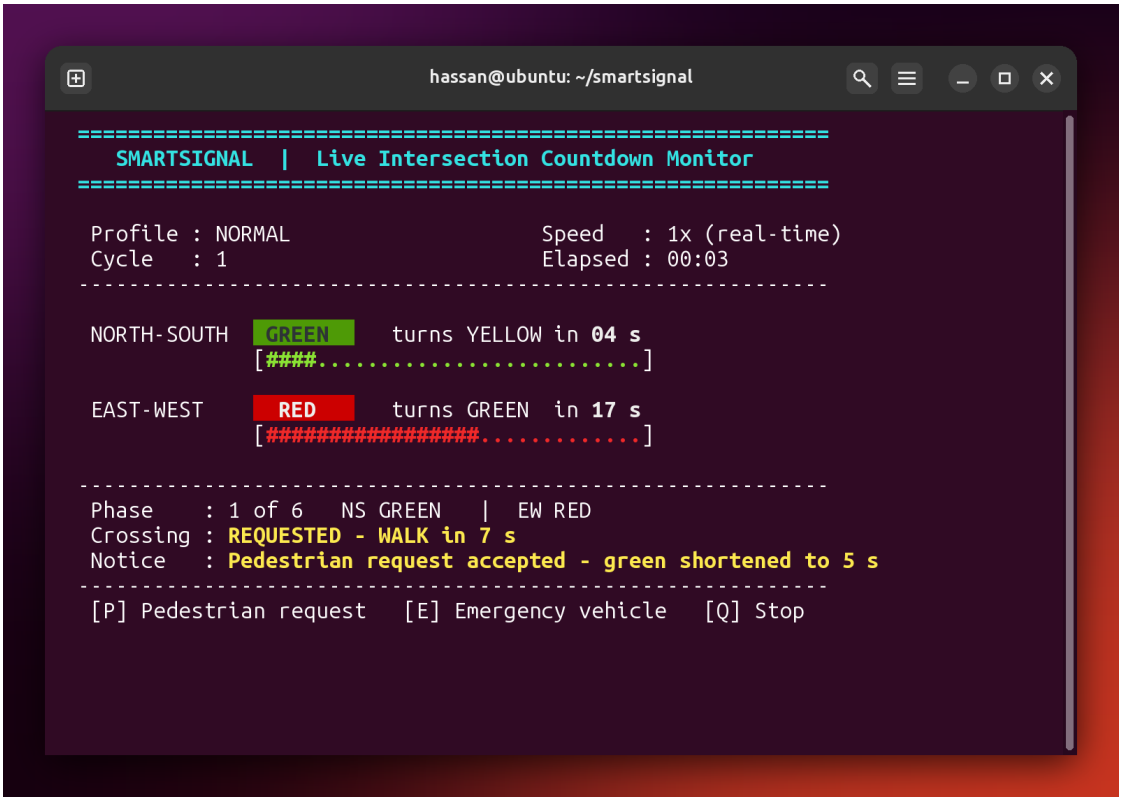


Figure 15: Pedestrian request accepted and green shortened to 5 seconds

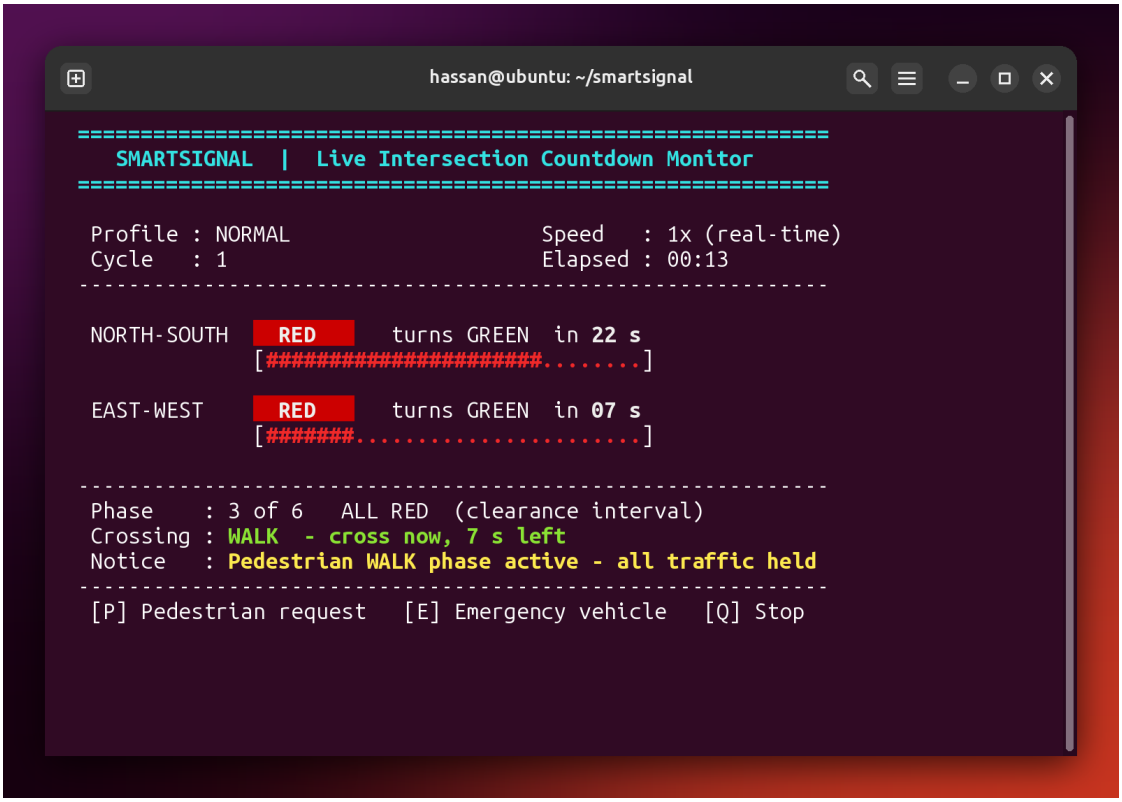


Figure 16: Pedestrian WALK phase during the all-red interval

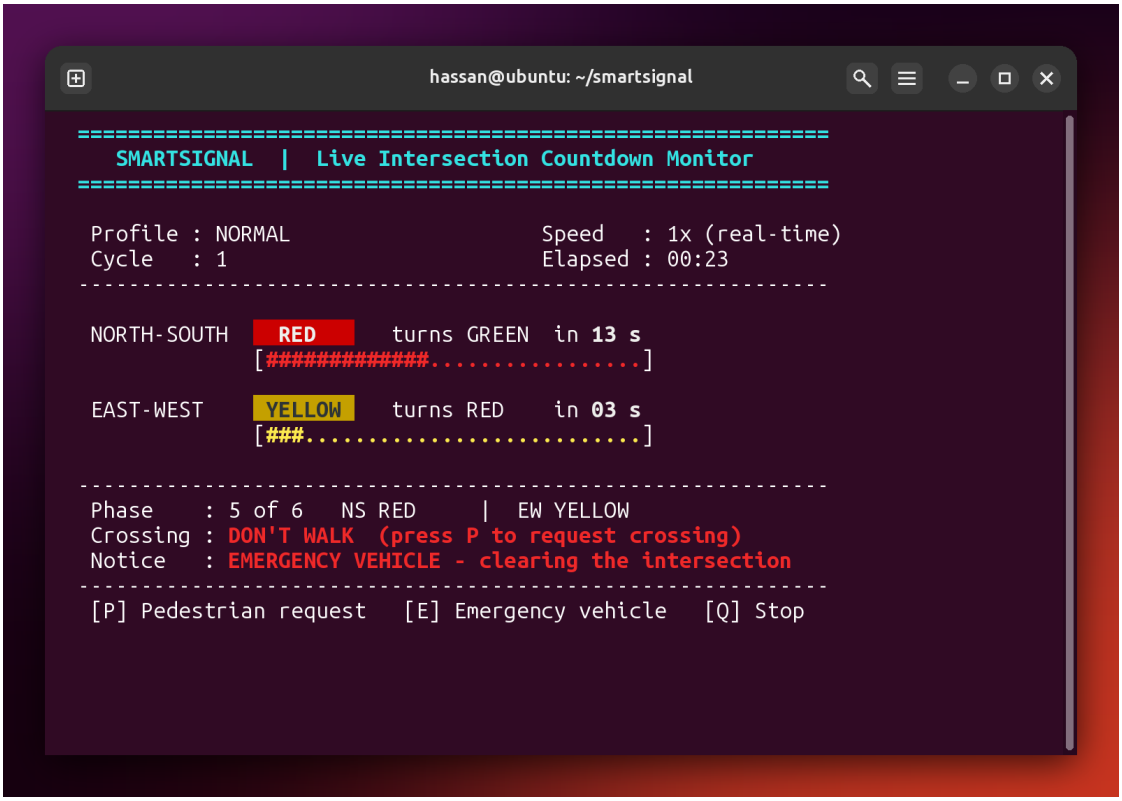


Figure 17: Emergency override: the green has ended and the intersection is clearing

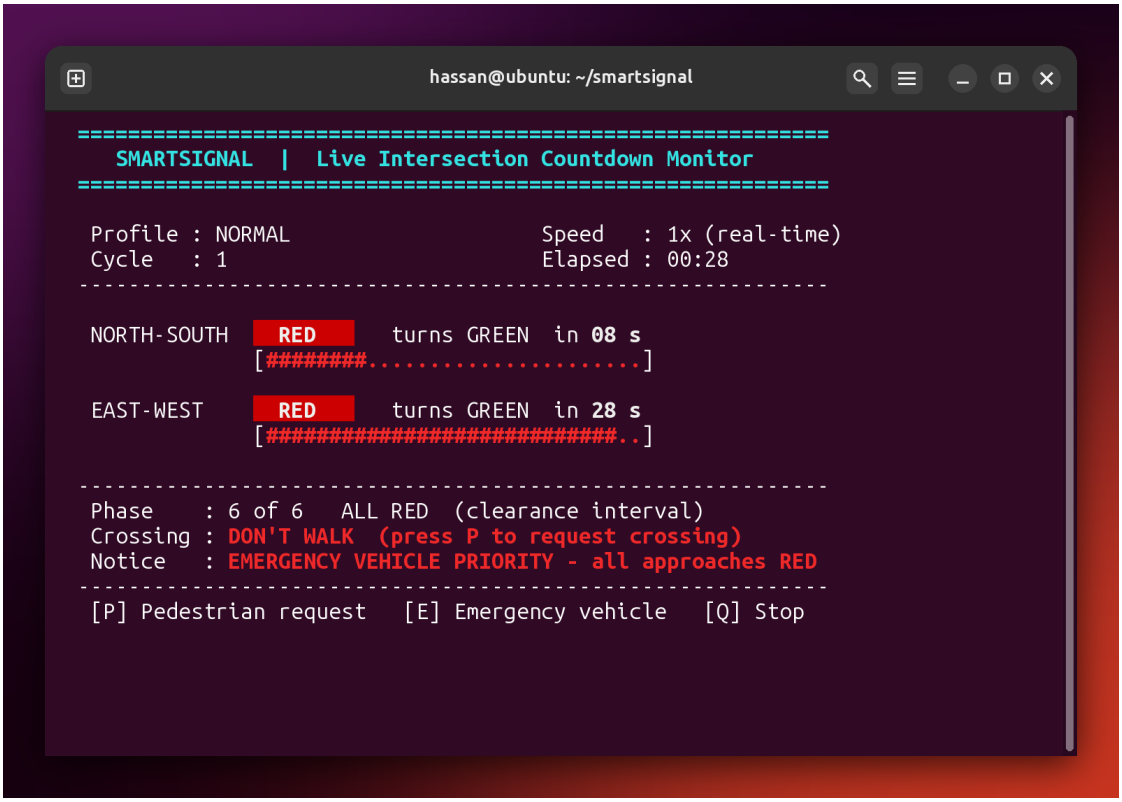


Figure 18: Emergency vehicle priority with all approaches held red

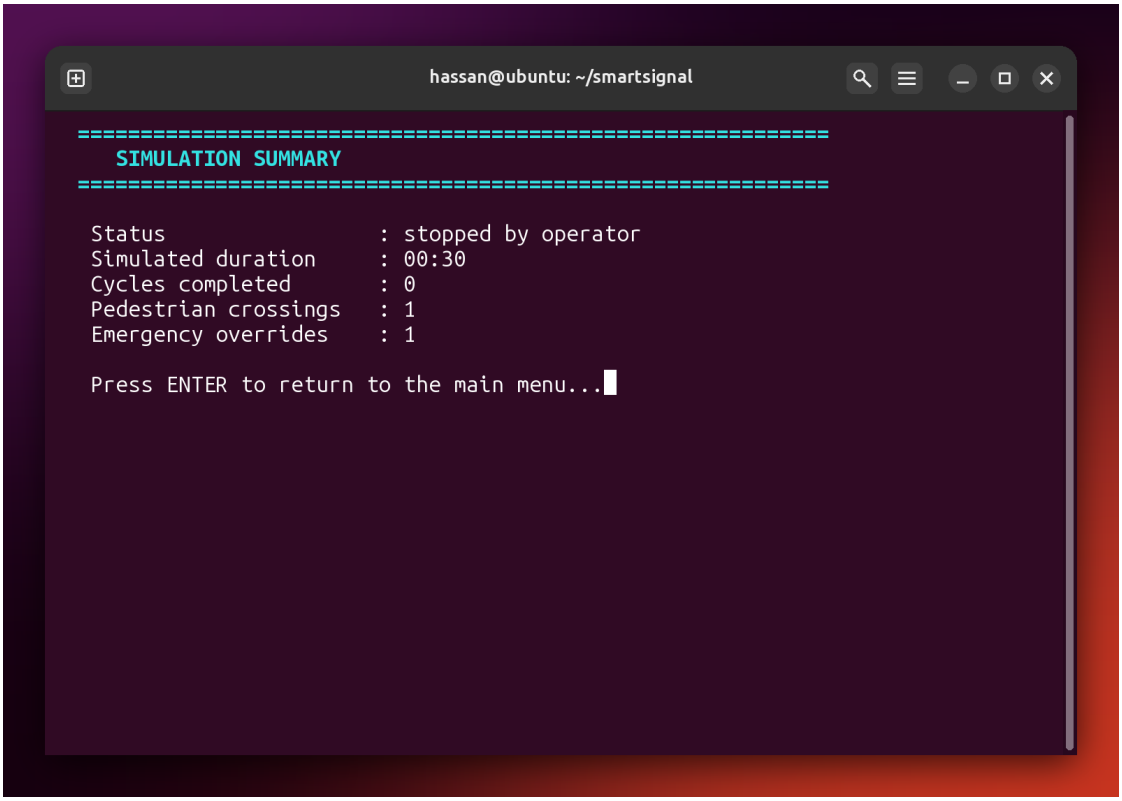


Figure 19: Simulation summary after the operator stops the simulation

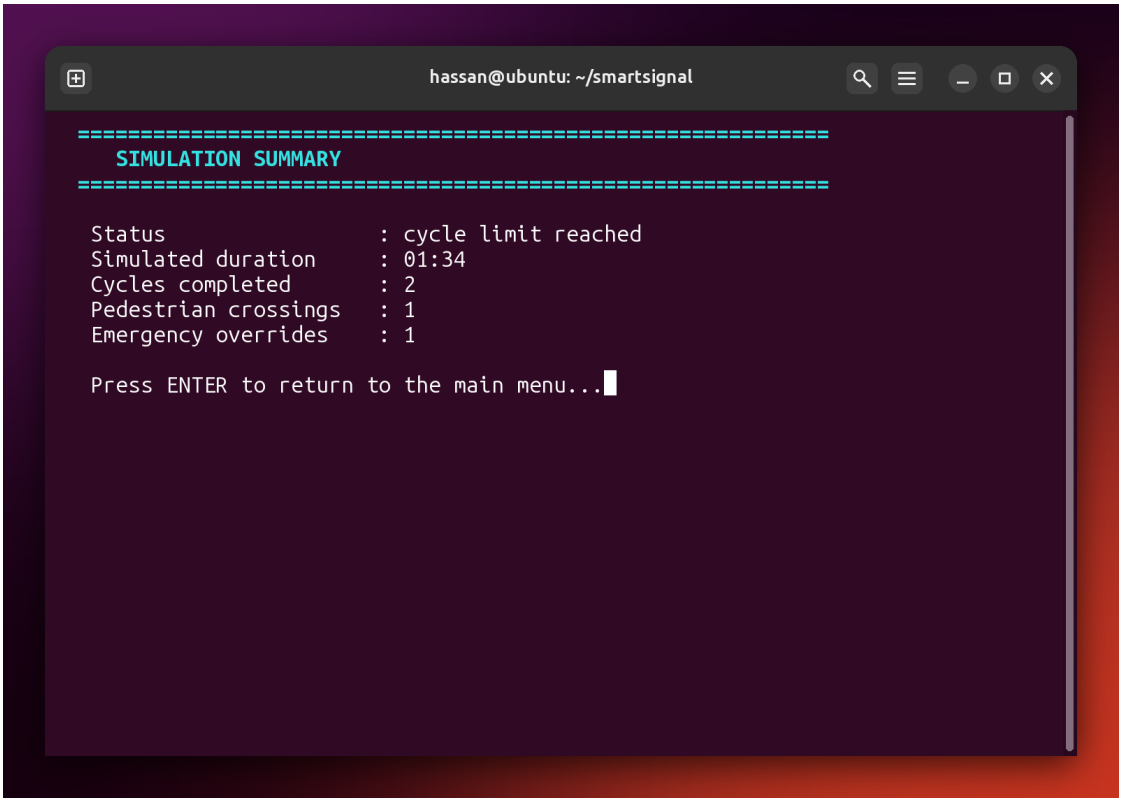


Figure 20: Simulation summary after the cycle limit is reached

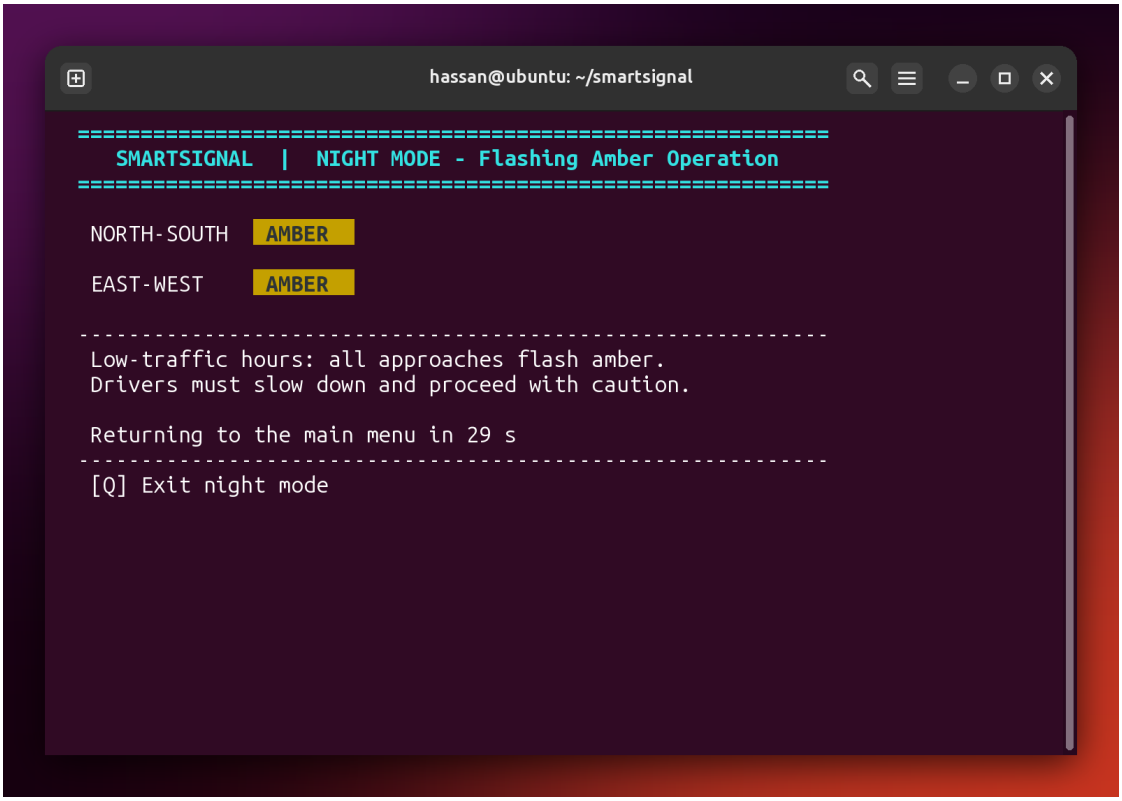


Figure 21: Night mode with the amber lamps lit

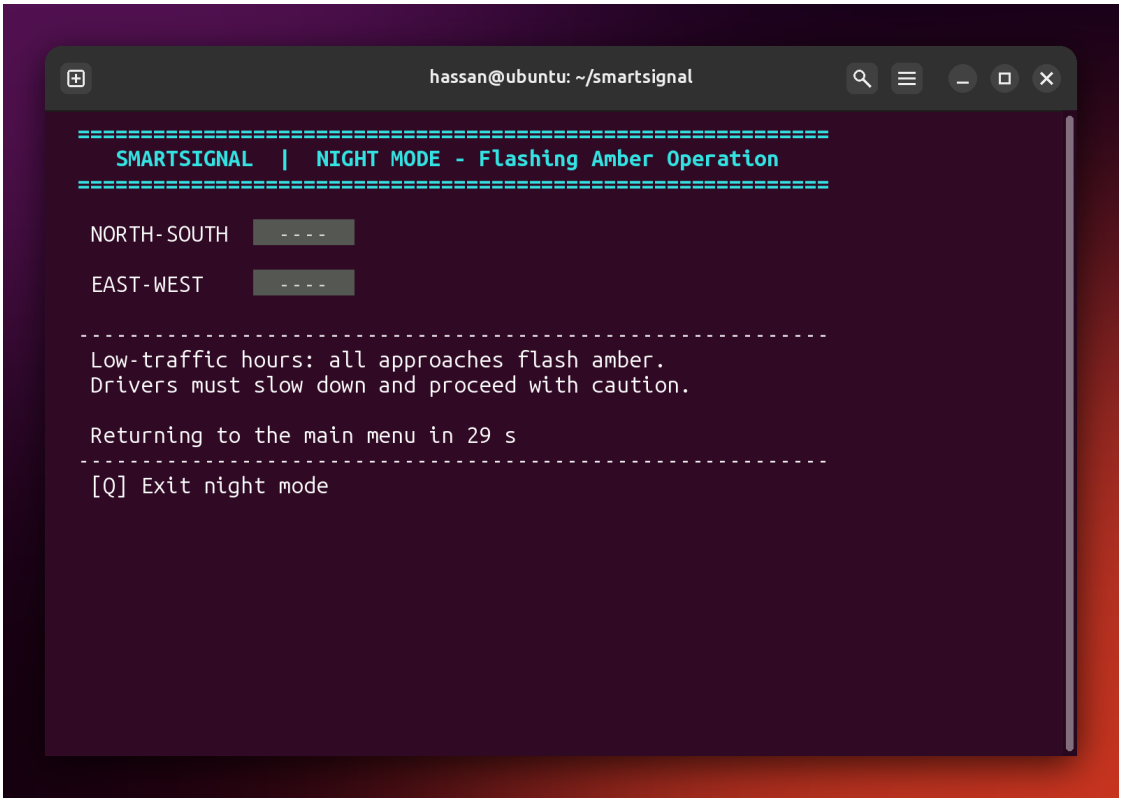


Figure 22: Night mode with the amber lamps off (flashing)

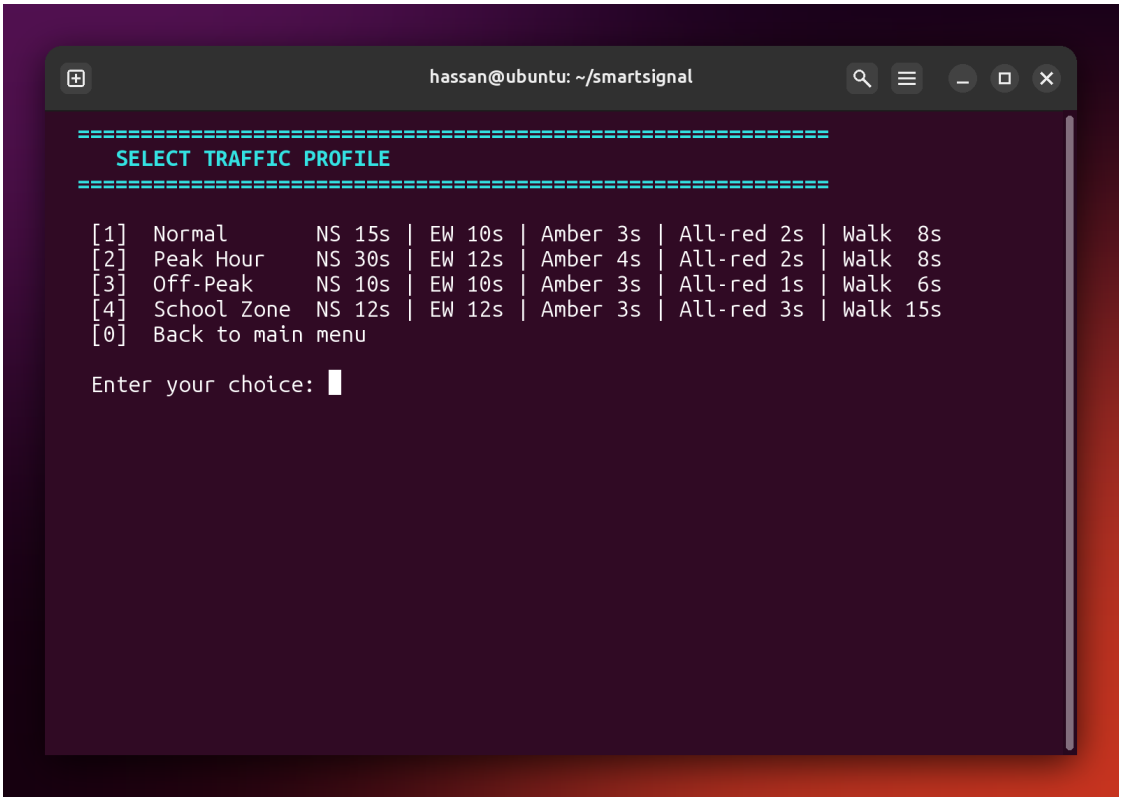


Figure 23: Traffic profile selection

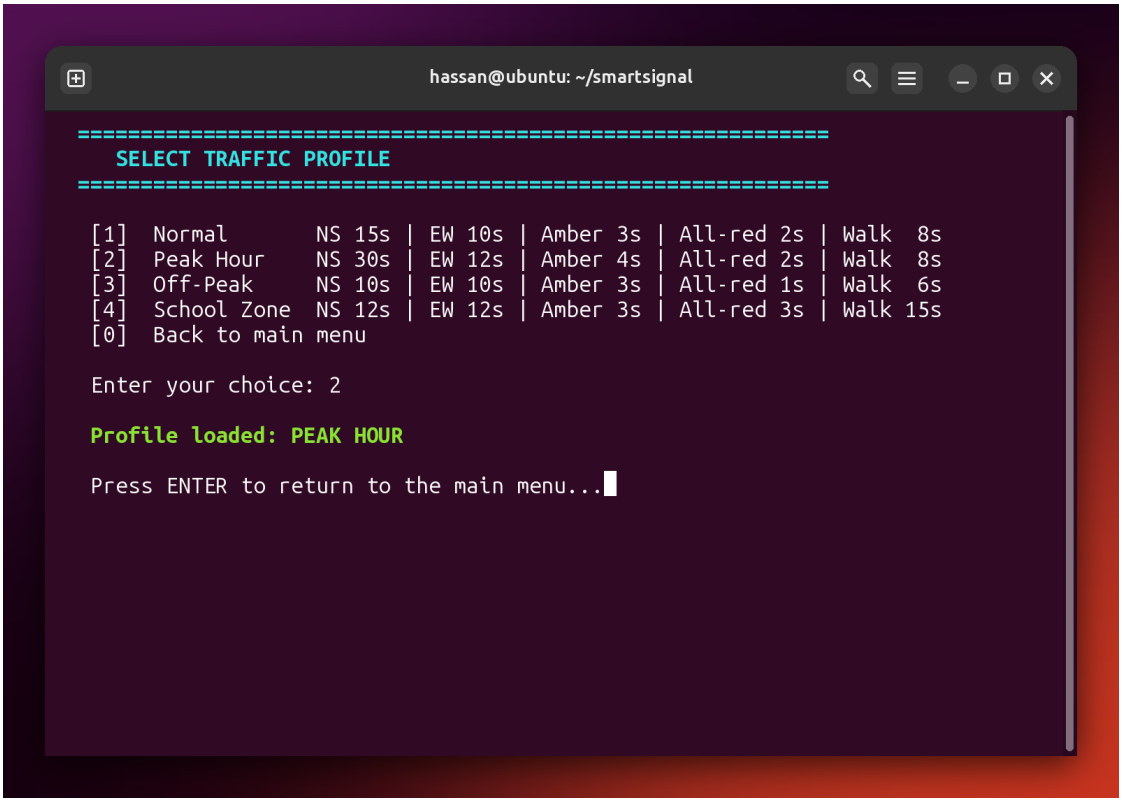


Figure 24: Peak Hour profile loaded

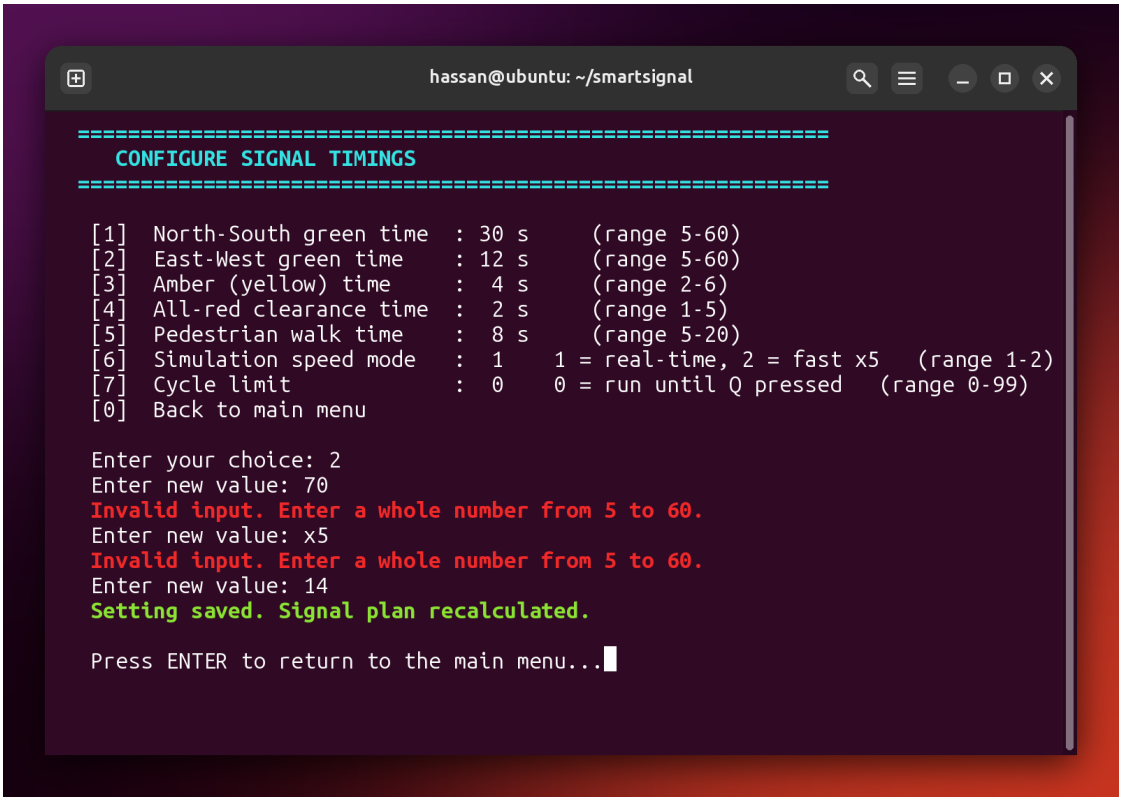


Figure 25: Configure timings with invalid input rejected

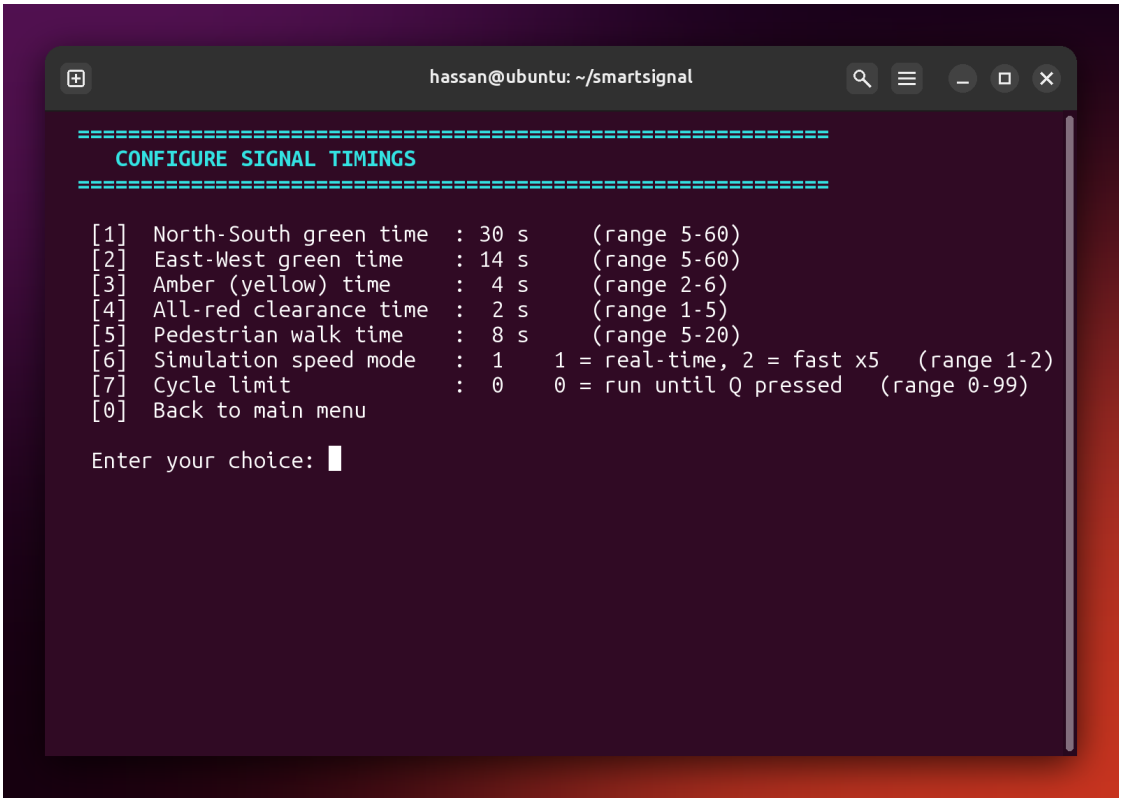


Figure 26: Settings after the East-West green time was changed

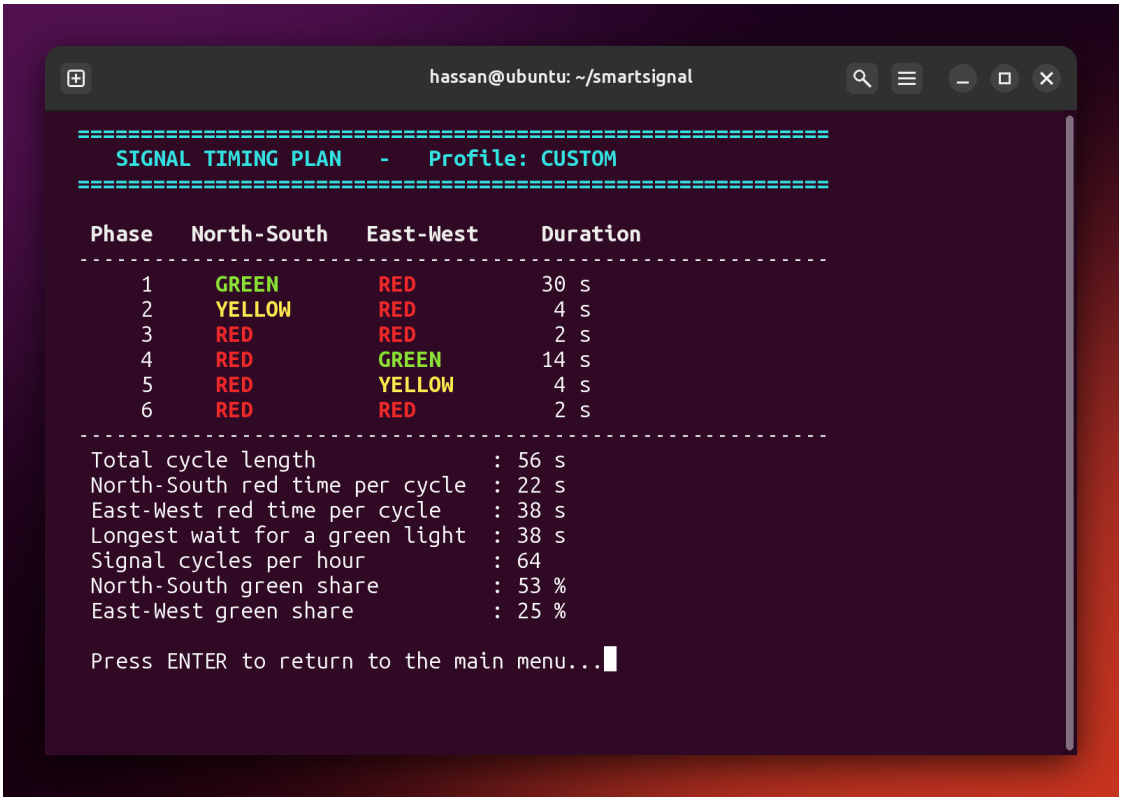


Figure 27: Signal timing plan calculated from the current settings

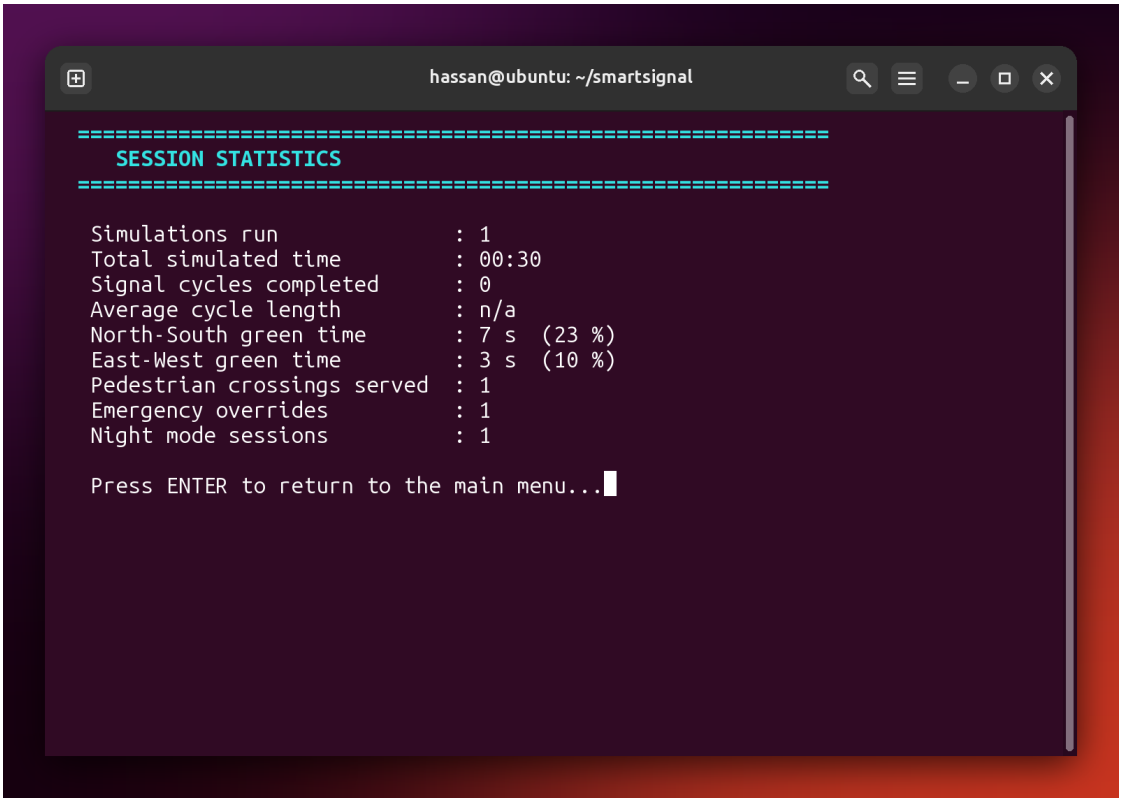


Figure 28: Session statistics

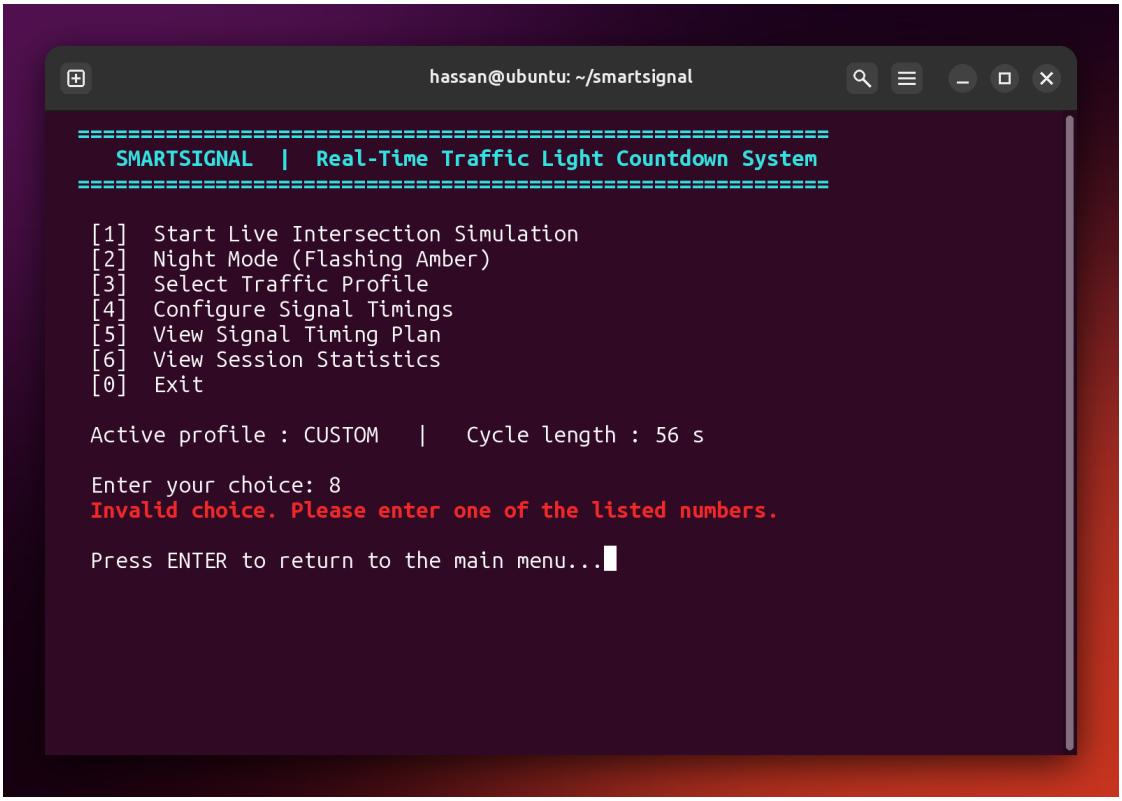


Figure 29: Invalid menu choice handling

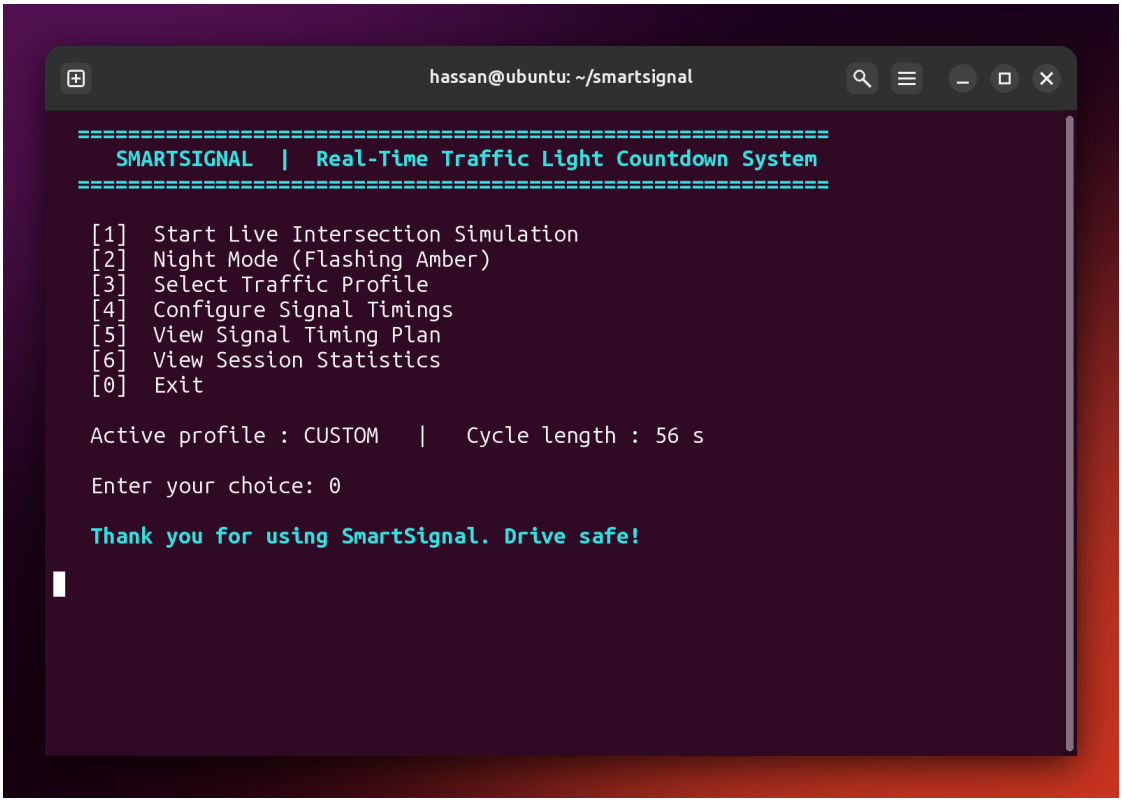


Figure 30: Exiting the program

7.0 Conclusion

The objective of this project was to create and implement a system that relies on x86 assembly language and employs arithmetic operations, looping, and jumping mechanisms. This target was accomplished by creating SmartSignal – a program that simulates the functioning of a traffic light at an intersection and implements a countdown – temp signal for each direction similar to the way modern traffic lights can work in navigation applications. Users could press the button for a pedestrian request or an emergency request during countdown execution, select a traffic profile, and enter timing values that would be validated, as well as analyze generated timings and statistics. ADD and SUB instructions were responsible for countdown, determining the time to change, for the timing calculations and statistics; LOOP and DEC/JNZ were in charge of executing timing loops, logic tables and display data; and conditional and unconditional jumps helped create user interface. The software builds without errors and managed to complete each of eighteen tests including edge tests such as overlapping requests, invalid input and Ctrl+C inputs during the program execution.

Analyses revealed that assembly language is still important within embedded systems, real-time systems, and operating system kernels, and it is necessary for cybercrime and forensics work. The reverse engineering exercise showed that even with no executable file information such as symbols or details about functions called, it was possible to reconstruct the use of program components from the simple strings present on the disassembled file.

The creation of the prototype has some drawbacks because it is limited to modeling one two-prong intersection; its clock occurs with software delays rather than hardware timer interruptions, and it runs only on 32-bit operating system. Future software improvements may include the interfacing of multiple intersections to achieve “green wave,” adjusting green time depending on

8.0 References

Antonakakis, M., April, T., Bailey, M., Bernhard, M., Bursztein, E., Cochran, J., Durumeric, Z., Halderman, J. A., Invernizzi, L., Kallitsis, M., Kumar, D., Lever, C., Ma, Z., Mason, J., Menscher, D., Seaman, C., Sullivan, N., Thomas, K., & Zhou, Y. (2017). Understanding the Mirai botnet. In *Proceedings of the 26th USENIX Security Symposium* (pp. 1093–1110). USENIX Association. https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/antonakakis

Barr, M., & Massa, A. (2006). *Programming embedded systems: With C and GNU development tools* (2nd ed.). O'Reilly Media.

Cloudflare. (n.d.). *What was the WannaCry ransomware attack?* Retrieved October 19, 2026, from https://www.cloudflare.com/learning/security/ransomware/wannacry-ransomware/

DH Web Desk. (2025, October 13). MapmyIndia's Mappls app now offers real-time traffic signal countdown feature in Bengaluru. *Deccan Herald*. https://www.deccanherald.com/technology/mapmyindias-mappls-app-now-offers-real-time-traffic-signal-countdown-feature-in-bengaluru-3762313

Duntemann, J. (2009). *Assembly language step-by-step: Programming with Linux* (3rd ed.). Wiley.

Eilam, E. (2005). *Reversing: Secrets of reverse engineering*. Wiley.

Falliere, N., O Murchu, L., & Chien, E. (2011). *W32.Stuxnet dossier* (Version 1.4). Symantec Security Response. https://docs.broadcom.com/doc/security-response-w32-stuxnet-dossier-11-en

FFmpeg. (n.d.). *FFmpeg assembly language lessons* [Source code repository]. GitHub. Retrieved October 19, 2026, from https://github.com/FFmpeg/asm-lessons

Godbolt, M. (n.d.). *Compiler Explorer*. Retrieved October 19, 2026, from https://godbolt.org/

Intel Corporation. (n.d.). *Intel 64 and IA-32 architectures software developer's manuals*. Retrieved October 19, 2026, from https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

Irvine, K. R. (2019). *Assembly language for x86 processors* (8th ed.). Pearson.

Kerrisk, M. (2010). *The Linux programming interface: A Linux and UNIX system programming handbook*. No Starch Press.

Ligh, M. H., Case, A., Levy, J., & Walters, A. (2014). *The art of memory forensics: Detecting malware and threats in Windows, Linux, and Mac memory*. Wiley.

National Academies of Sciences, Engineering, and Medicine. (2015). *Signal timing manual* (2nd ed., NCHRP Report 812). The National Academies Press. https://doi.org/10.17226/22097

National Security Agency. (2019, March 5). *Ghidra – the software reverse engineering tool you've been waiting for – is here!* https://www.nsa.gov/Press-Room/News-Highlights/Article/Article/1775584/ghidra-the-software-reverse-engineering-tool-youve-been-waiting-for-is-here/

National Security Agency. (2023, March 6). *Four years later: The impacts of Ghidra's public release*. https://www.nsa.gov/Press-Room/News-Highlights/Article/Article/3319971/four-years-later-the-impacts-of-ghidras-public-release/

Proven, L. (2022, October 29). RIP: Kathleen Booth, the inventor of assembly language. *The Register*. https://www.theregister.com/2022/10/29/kathleen_booth_obit/

Renesis, A. (2026, September 23). Google Maps, Waze & Apple Maps put to shame by Chinese navigation app which shows live countdowns for traffic lights you haven't reached yet. *Supercar Blondie*. https://supercarblondie.com/google-maps-waze-apple-maps-debate-chinese-navigation-app-amap/

Shilov, A. (2024, November 4). FFmpeg devs boast of up to 94x performance boost after implementing handwritten AVX-512 assembly code. *Tom's Hardware*. https://www.tomshardware.com/pc-components/cpus/ffmpeg-devs-boast-of-up-to-94x-performance-boost-after-implementing-handwritten-avx-512-assembly-code

Sikorski, M., & Honig, A. (2012). *Practical malware analysis: The hands-on guide to dissecting malicious software*. No Starch Press.

The Linux Kernel Organization. (n.d.). *Linux kernel source tree: arch/x86/entry* [Source code]. GitHub. Retrieved October 19, 2026, from https://github.com/torvalds/linux/tree/master/arch/x86/entry

The NASM Development Team. (n.d.). *NASM: The Netwide Assembler documentation*. Retrieved October 19, 2026, from https://www.nasm.us/docs.php

VirusTotal. (n.d.). *Writing YARA rules*. YARA documentation. Retrieved October 19, 2026, from https://yara.readthedocs.io/en/stable/writingrules.html

Yiu, J. (2013). *The definitive guide to ARM Cortex-M3 and Cortex-M4 processors* (3rd ed.). Newnes.

Appendix A: AI Usage Declaration

Tool used: Claude (Anthropic),Chat-GPT, accessed via the web interface.

What it was used for: Generative AI was used only as a supporting tool during this assignment. Specifically, it was used to:

- Think of several potential system ideas and compare them before finalizing the idea of the traffic-light countdown system.
- Assist with the location of various sources for the background study required for the research process.
- Support in the process of debugging and testing certain parts of the NASM code.
- Help to compile and check drafts of the report and design diagrams.

What I accomplished personally: The choice of the SmartSignal idea was mine, along with the specification of its characteristics and requirements. I developed the logic of traffic phases, countdown calculations, pedestrian and emergency rules, and the overall structure of the program; I created and tested the loading process of the program. I conducted testing and reverse engineering, created screenshots, and made the final decisions about the contents. All AI recommendations were reviewed and edited before being used in the project. All sources have been checked and cited in the References section.

Declaration: I hereby declare that all information above is true and complete account of how the generative AI has been used in this work as well as that the whole system and report is the product of my own work.

Name: Muhammad Hassan

Date: 26th/Oct/2026

B.1 Summary table

No.	Date	Purpose	Prompt (short)	What the AI produced	How it was used
1	20-Sep-2026	Choosing system idea	Asked for assembly project ideas that use ADD/SUB, loops and jumps	A list of possible systems	Used to brainstorm; I chose the traffic-signal idea
2	22-Sep-2026	Research sources	Asked for sources on assembly use in security and forensics	Suggested readings and case studies	I located and verified each source myself
3	28-Sep-2026	Debugging	Asked why a loop was not terminating	Explanation of flag behaviour and DEC/JNZ	Adapted the fix into my own code
4	5-Oct-2026	Report structure	Asked how to organise the implementation section	Suggested section headings	Used as a guide only; I wrote the content
5	12-Oct-2026	Flowchart concepts	Asked how to describe the pedestrian/emergency state transitions	Text description of transitions	I drew the diagrams myself from this description

B.2 Evidence

Prompt 1: 20 September 2026 – Choosing system idea

- **My prompt**: "I am stuck on how to go about implementing a "cat" command in assembly programming, please provide me with useful ideas to help me get started."
- **AI response**: Gave clear steps on how to make use of the “cat” command and explained how to do it in assembly language.
- **How I used it**: I found the suggestions and incorporated them into writing my code.

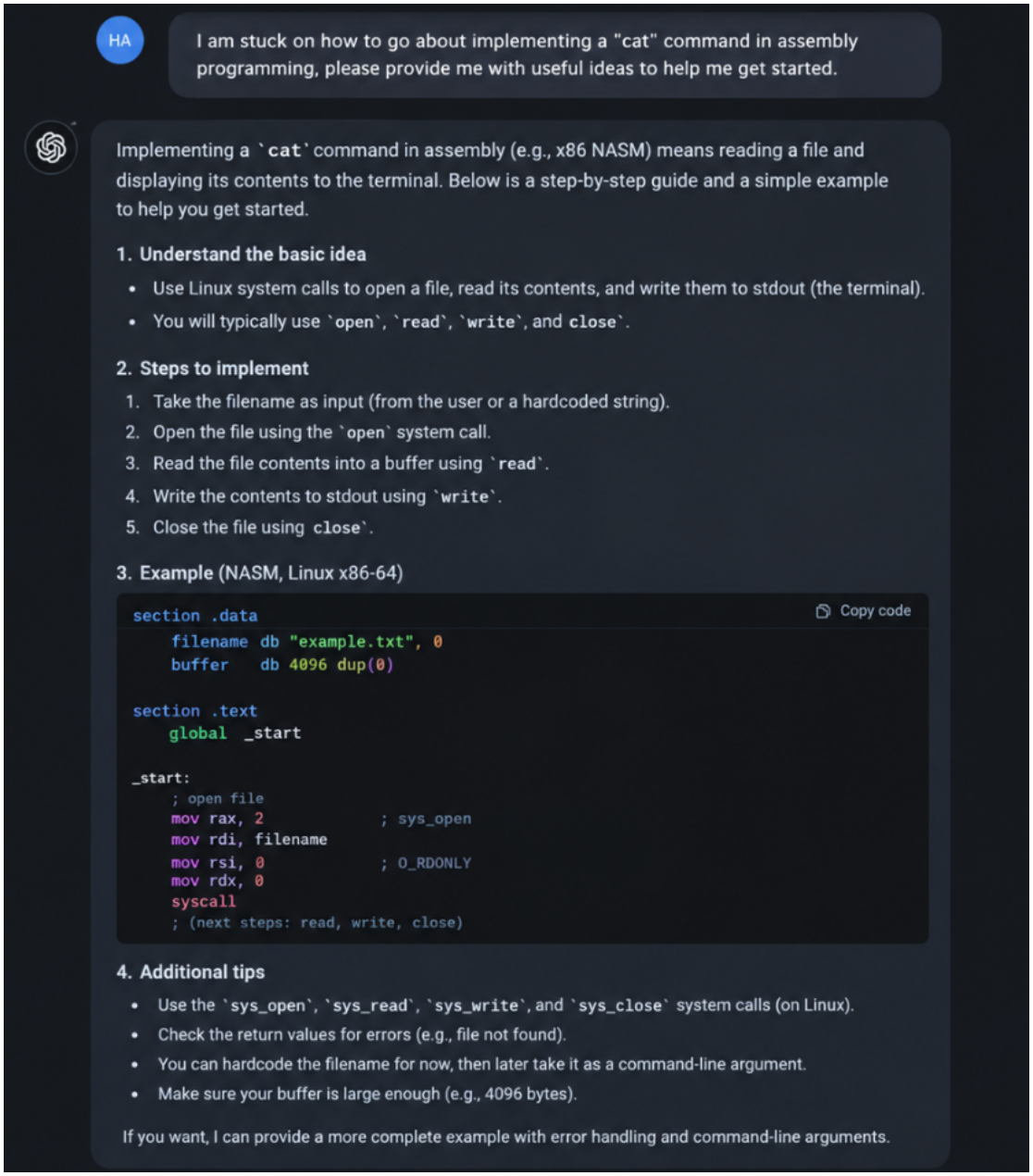


Figure B1: Prompt and response for implementation of "cat" command in assembly programming

Prompt 2: 22 September 2026 – Research sources

- **My prompt:** "Can you point me to credible sources on where assembly language is still used, especially in malware analysis and digital forensics?"
- **AI response:** Listed suggested readings including Sikorski & Honig (2012), Eilam (2005) and sources on Ghidra and FFmpeg.

- **How I used it:** I found and checked each source myself and cited them in APA style.

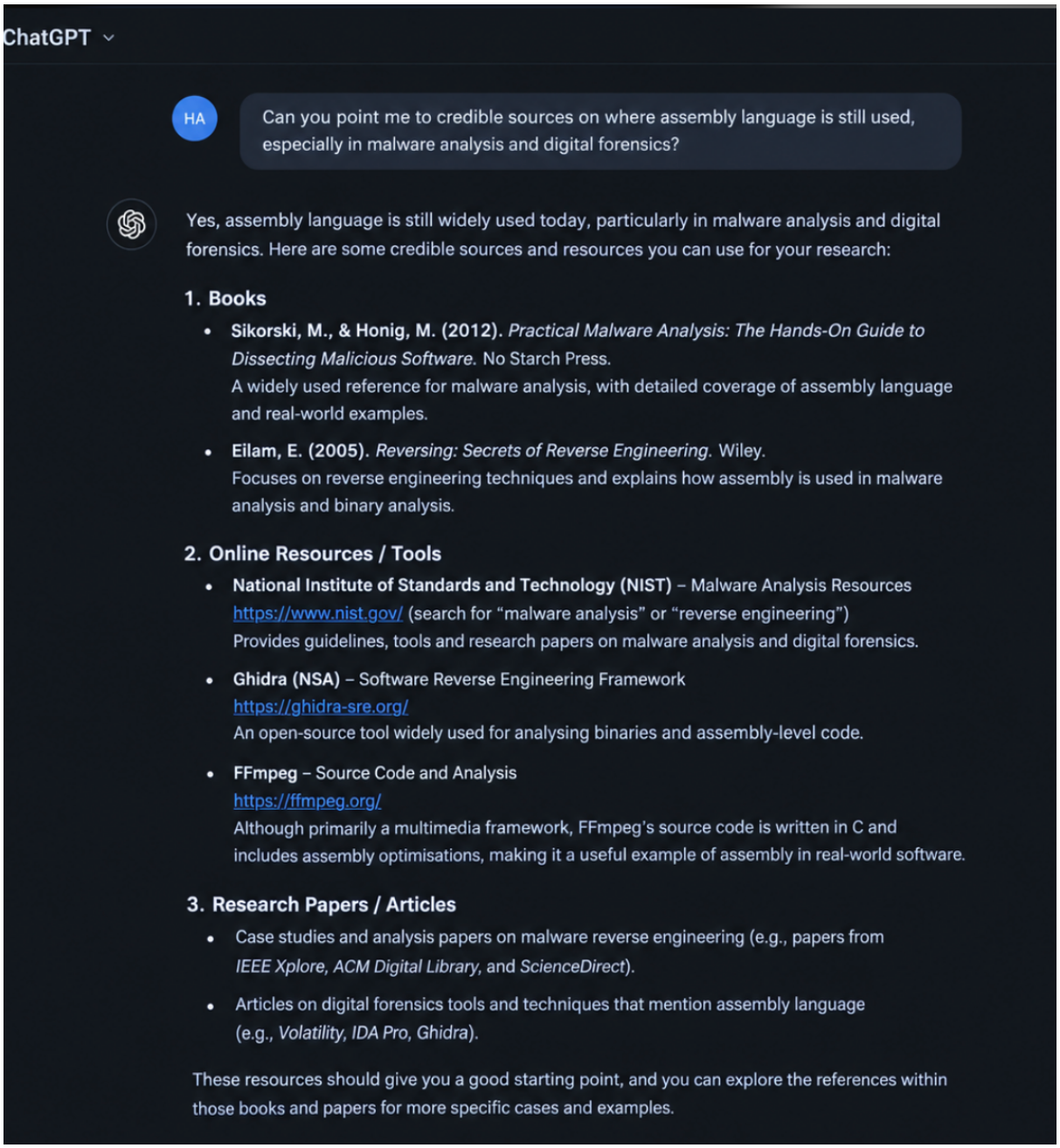


Figure B2: Prompt and response for research sources

Prompt 3: 28 September 2026 – Debugging

- **My prompt:** "My countdown loop using DEC and JNZ keeps running forever. What could cause that?"

- **AI response:** Explained that a flag or counter was likely being overwritten before the DEC/JNZ check, and suggested checking register usage.
- **How I used it:** I traced my code, found the counter was being clobbered, and fixed it myself.

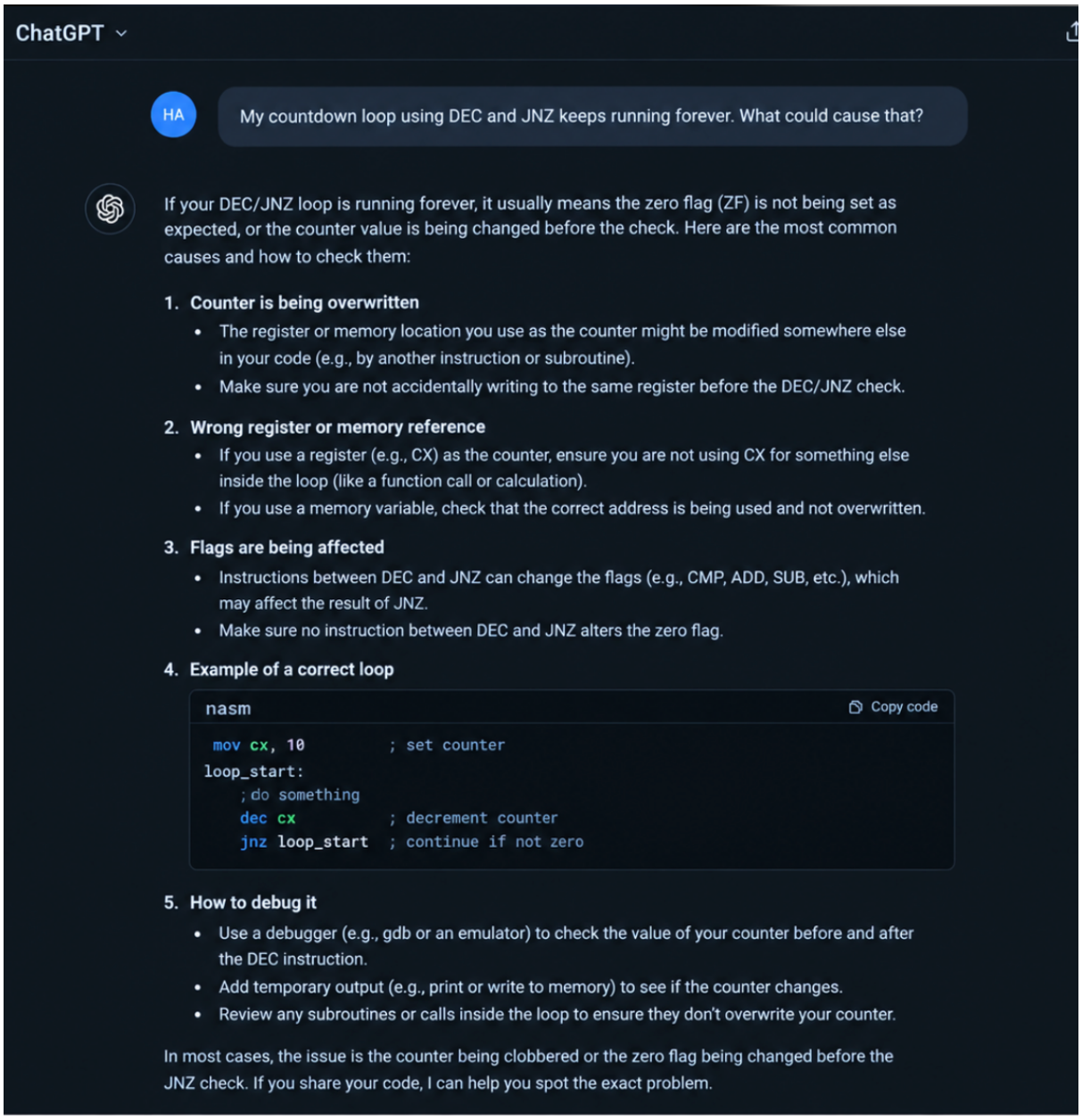


Figure B3: Prompt and response for debugging the loop

Prompt 4: 5 October 2026 – Report structure

- **My prompt:** "What is a sensible structure for the implementation section of an assembly report?"
- **AI response:** Suggested headings such as program structure, arithmetic, loops, jumps and testing.
- **How I used it:** Used only as a heading guide; I wrote all the content and explanations myself.

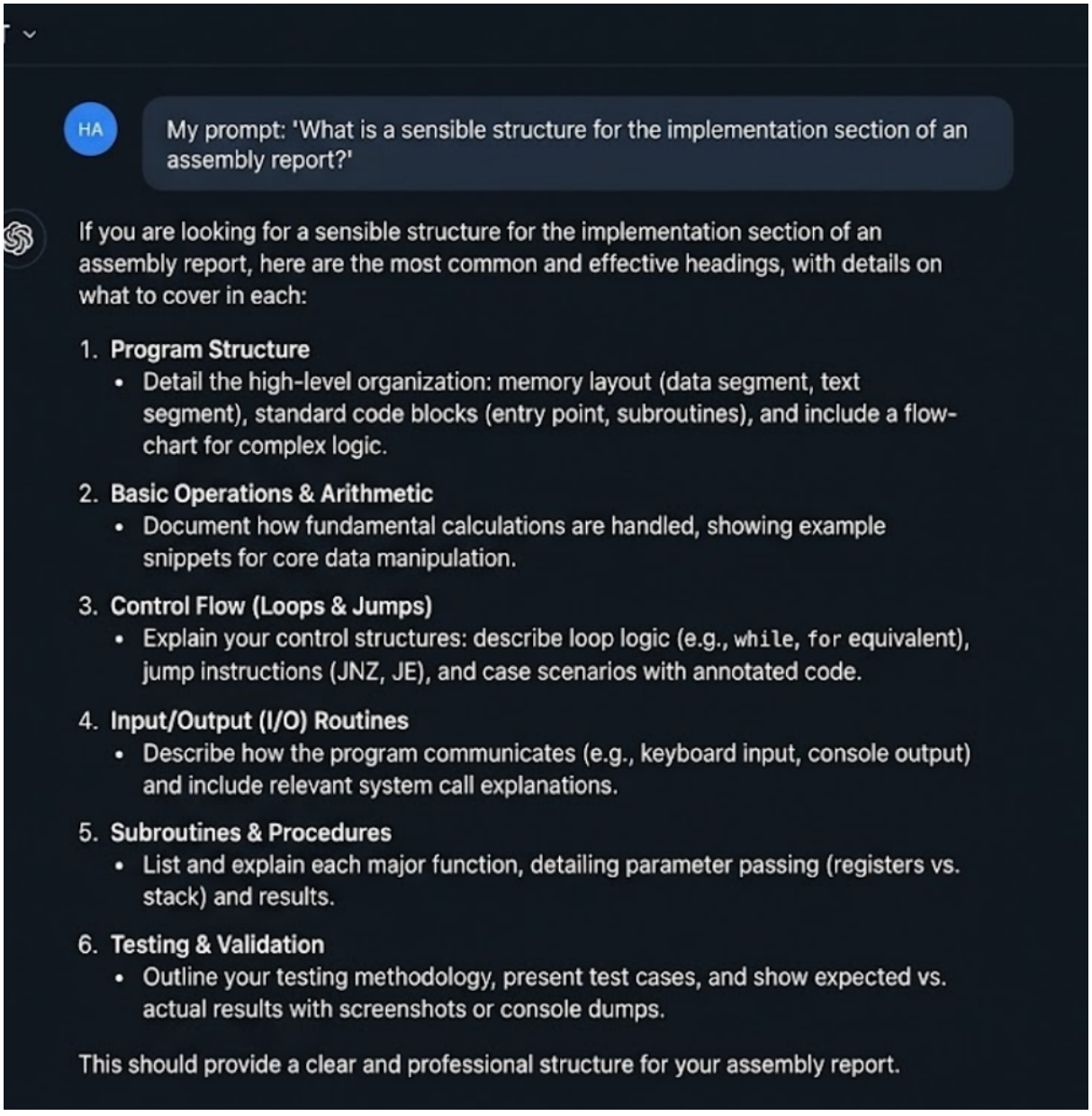


Figure B4: Prompt and response for report structure

Statement

The generative AI tools were only used in this assignment to help brainstorm ideas, source suggestions, debug support, and structure ideas. All of the content, codes, analyses, and diagrams that were produced were created by me; I produced, reviewed, and edited them. Not declaring the use of AI may be treated as an act of academic misconduct.